
Return-to-Go Is More Than a Number: Q-Guided Alignment for Return-Conditioned Supervised Learning

Yuxiao Yang¹ Weitong Zhang¹

Abstract

Conditioned Sequence Models (CSMs) learn policies by treating return-to-go (RTG) as a control signal. However, existing CSMs often treat the RTGs as simple numerical inputs rather than aligning them with the performance of their policies. In this paper, we propose Q-ALIGN DT, a framework that enforces this alignment by ensuring the Q -value of the output policy is consistent with the input RTG. By leveraging a Q function to provide dense guidance to CSMs and further fine-tuning it using an *RTG-perturbation* technique with the CSM, our method ensures that higher RTGs are consistently mapped to trajectories with higher expected returns. Theoretically, we show that Q-ALIGN DT can efficiently learn the desired policy and output a near-optimal one when the RTG is sufficiently high. Empirically, we demonstrate through extensive experiments that Q-ALIGN DT achieves superior controllability and performance across the D4RL benchmark. Remarkably, our model effectively learns a structured family of policies that maintains precise alignment and generalizes to tasks like velocity-tracking where prior methods fail.

1. Introduction

Offline reinforcement learning (Offline RL) aims to learn an effective and robust policy that can be deployed without interacting with the environment, relying solely on pre-collected datasets (Fujimoto et al., 2019). Recently, transformer architectures (Vaswani et al., 2017), which have shown remarkable success in natural language processing (Devlin et al., 2019; Brown et al., 2020; Liu et al., 2019) and computer vision (Dosovitskiy et al., 2021a; He et al., 2021;

Dosovitskiy et al., 2021b; Liu et al., 2021), have also been adopted in RL due to their powerful sequence modeling capabilities (Laskin et al., 2023; Lee et al., 2023; Grigsby et al., 2024). Among these advances, Conditional Sequence Models (CSMs) (Chen et al., 2021; Janner et al., 2021) provide a new perspective by treating policy learning as a supervised sequence modeling problem conditioned on the desired performance. In particular, Decision Transformer (DT) introduces a Return-to-Go (RTG) token which enables the model to generate trajectories that achieve expected returns rather than imitating the behavioral policy.

However, how the actual return obtained by a CSM *aligns* with the target input RTG is a fundamental yet often overlooked property. Precise alignment enables a single model to represent a diverse family of policies and is essential for controllable robot behaviors with varying velocities (Tanaka et al., 2025). Therefore, we would like to ask:

How well can CSMs actually align with target RTGs?

Unfortunately, recent studies (Kim et al., 2024a; Tanaka et al., 2025) report that existing CSMs often exhibit significant insensitivity to RTG and fail to achieve proper alignment. As empirically demonstrated in Figure 1, we hypothesize that this failure stems from a lack of structural awareness: a robust CSM should capture the structure between different RTG targets and their corresponding behaviors instead of merely treating the RTGs as simple tokens. Specifically, a higher RTG should consistently correspond to trajectories with higher expected cumulative returns, ensuring this *partial order* between target RTGs and realized performance.

In offline RL, enforcing such a partial order is challenging since it can be infeasible to construct sufficient trajectories within the fixed dataset following the partial order. To address this challenge, in this paper, we propose Q-ALIGN DT, which introduces an auxiliary Q -function to provide dense guidance. With a novel *RTG-to-behavior* objective alongside an *RTG-perturbation* technique, our method encourages the model to output actions that precisely reflect the relative differences in desired RTGs. We further integrate this RTG perturbation into the Q -function updates during co-training, ensuring that the critic and the policy

¹University of North Carolina at Chapel Hill. Correspondence to: Yuxiao Yang <yxyang@unc.edu>, Weitong Zhang <weitongz@unc.edu>.

co-evolve toward a consistent, reward-sensitive behavior.

Building on our algorithm, we provide a theoretical analysis of the alignment properties of CSMs and conduct extensive experiments to evaluate our trained models. We find that Q-ALIGN DT learns a family of RTG-conditioned policies that actively respond to target shifts, rather than merely relying on static reward associations found in the training data. Moreover, we demonstrate that our model can be adapted to distinct tasks (e.g., *HalfCheetah-Vel*) while maintaining competitive performance and alignment, indicating its potential for generalization across diverse behaviors.

Overall, our contributions are threefold:

- We propose Q-ALIGN DT, which introduces a new RTG-to-behavior alignment objective to enforce consistency between the input RTG and policy behavior, significantly improving RTG-conditioned alignment. It further employs a co-training framework with RTG perturbations that provide a high-quality action space for Q -function learning, enabling bidirectional improvements.
- Theoretically, we show that Q-ALIGN DT improves alignment by restricting the policy class, and that the alignment objective ensures, under high-RTG conditioning, equivalence to maximizing the Q -function. Combined with RTG perturbations, this leads to convergence to a near-optimal in-distribution policy under mild assumptions.
- Extensive experiments show that Q-ALIGN DT consistently achieves competitive performance across a wide range of offline RL tasks while significantly improving RTG-conditioned alignment. Remarkably and somewhat surprisingly, we report that Q-ALIGN DT generalizes effectively to the challenging *HalfCheetah-Vel* task and attains competitive performance by only controlling the RTG signal with zero-shot transfer.

Code. Our code is available at <https://github.com/yangyuxiao-sjtu/Q-Align-DT>.

2. Related Works

2.1. Offline Reinforcement Learning

Reinforcement Learning aims to train an agent to solve tasks through direct interaction with the environment, but such interaction is often expensive or impractical in domains such as robotics and healthcare. To address this limitation, Offline Reinforcement Learning learns policies from a fixed dataset collected by a behavior policy (Levine et al., 2020; Siegel et al., 2020; Jaques et al., 2019; Agarwal et al., 2020; Ernst et al., 2005).

Despite its promise, naively applying online RL algorithms in an offline setting often leads to severe performance degradation due to out-of-distribution actions and extrapolation errors (Fujimoto et al., 2019; Kumar et al., 2019; Levine

et al., 2020). Existing work mitigates these issues through a range of techniques, including Q -value regularization (Wu et al., 2019; Kumar et al., 2020; Wang et al., 2020) and behavior cloning-based objectives (Fujimoto & Gu, 2021).

More recently, the fixed-dataset nature of Offline RL has motivated the adoption of Transformer-based architectures (Chen et al., 2021; Wu et al., 2023; Chebotar et al., 2023), which enable efficient and fully parallelized supervised training over trajectories.

2.2. Conditioned Sequence Models

Conditioned Sequence Models (CSMs) (Chen et al., 2021; Janner et al., 2021) cast RL in the supervised learning paradigm by predicting actions from historical state-action pairs (s_i, a_i) and return-to-go (RTG) tokens rtg_i . In particular, the RTG is derived from the dataset as cumulative rewards during training, and is provided as a user-specified conditioning during inference. Although recent theoretical results suggest CSMs can recover target returns under ideal assumptions (Brandfonbrener et al., 2022; Lin et al., 2024; Furuta et al., 2022), empirical studies (Kim et al., 2024a; Tanaka et al., 2025) show that the information carried by the RTG is often under-utilized by CSMs, leading to poor alignment between the target behavior and desired RTGs.

To address this issue, RADT (Tanaka et al., 2025) enhances RTG sensitivity through architectural modifications, at the cost of substantial computational and parameter overhead, by introducing an additional attention layer in each transformer block.

2.3. Q -Learning in Conditioned Sequence Models

Q -functions are widely used to improve CSMs. Early methods like QDT (Yamagata et al., 2023) and CGDT (Wang et al., 2024) leverage pretrained Q -functions for RTG relabeling and dataset-bias mitigation, while later approaches such as QT (Hu et al., 2024), QCS (Kim et al., 2024b), and TD3-ODT (Yan et al., 2024) backpropagate Q -value gradients through predicted actions.

While Q -gradient methods achieve competitive peak performance, they often push the policy toward maximum-value actions regardless of the target RTG, degrading alignment as it collapses to relatively high-return regions within the data distribution (Figure 1). Furthermore, although existing methods leverage Q -functions to improve CSMs, little attention has been paid to how CSMs can, in turn, benefit Q -function training through their alignment capabilities.

2.4. Multi-Task and Meta-Reinforcement Learning

Meta-Reinforcement Learning (Meta-RL) (Beck et al., 2025) aims to quickly adapt an agent to new tasks with

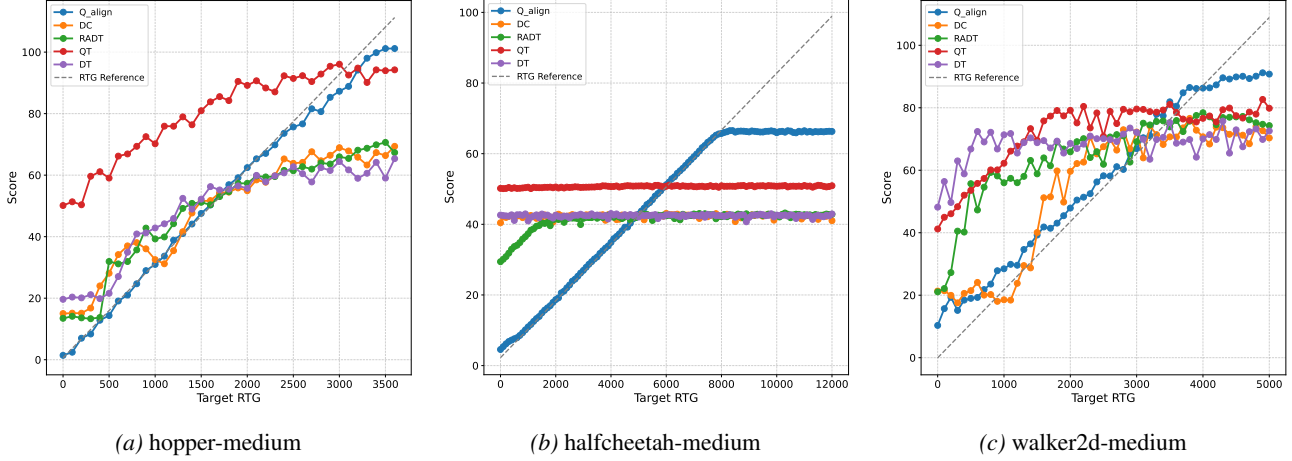


Figure 1. Performance of Q-ALIGN DT (Q-align) and other baseline models on D4RL tasks. Target RTGs are set with an interval of 100 targeting for cumulative rewards. We sample 30 trajectories for each target RTG and report the mean performance for each method.

similar underlying structures (Finn et al., 2017; Duan et al., 2016). A more challenging setting, Offline Meta-RL, considers scenarios where the agent must learn from a fixed dataset and is expected to generalize to unseen test tasks (Mitchell et al., 2021; Dorfman et al., 2021). Owing to the in-context learning capabilities of Transformer architectures, recent works have increasingly deployed Transformers for such tasks (Xu et al., 2022; Lee et al., 2023).

3. Preliminaries

We consider a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ and $\mathcal{A}$ represent the state and action spaces, $P(s_{t+1}|s_t, a_t)$ denotes the transition probability, $R(s, a)$ is the reward function, and $\gamma \in (0, 1]$ is the discount factor. Following the Decision Transformer framework (Chen et al., 2021), we represent the input as a sequence of reward-to-go (RTG), state, and action tokens:

$$\tau = (\text{rtg}_0, s_0, a_0, \dots, \text{rtg}_H, s_H, a_H),$$

where $\text{rtg}_t = \sum_{i=t}^H r_i$ represents the cumulative future reward at time t . This RTG signal serves as the target return to condition the policy. To ensure computational efficiency, we employ a k -step context window, where the truncated sequence at time t is defined as:

$$\tau_t = (\text{rtg}_{t-k+1}, s_{t-k+1}, a_{t-k+1}, \dots, \text{rtg}_t, s_t, a_t).$$

The RTG token serves as the primary mechanism for conditioning the model’s behavior. Ideally, a DT-based model should represent a family of policies indexed by the target RTG (Brandfonbrener et al., 2022), expressed as:

$$\Pi_{\text{DT}} = \{\pi_z \mid z \in \mathbb{R}_+\}, \quad (1)$$

In practice, the RTG tokens are usually controlled during inference to inform the desired behavior. To reflect this, we

define a *modified RTG sequence* τ_t^g for any scalar $g \in \mathbb{R}$ by shifting all RTG tokens in the context window by g :

$$\tau_t^g = (\text{rtg}_{t-k+1} + g, s_{t-k+1}, a_{t-k+1}, \dots, \text{rtg}_t + g, s_t, a_t). \quad (2)$$

4. Methods

Motivation. To investigate the interpretability of target RTG, we benchmark several variants of the Decision Transformer (Chen et al., 2021; Hu et al., 2024; Tanaka et al., 2025; Kim et al., 2024a) and analyze their behavior under various RTG conditioning values. As illustrated in Figure 1, we consistently observe a significant gap between the target RTG and the actual rollout performance.

This misalignment is especially evident in the *HalfCheetah* environment, where the model exhibits almost no sensitivity to variations in the requested RTG. Such findings suggest that the RTG conditioning remains marginal in the model’s decision process, directly motivating the development of Q-ALIGN DT to explicitly enforce RTG-behavior alignment.

4.1. Training Model with RTG-to-Behavior Alignment

From the above discussion, we aim to train a CSM variant that internalizes the **partial ordering between return-to-go (RTG) and action quality**. Ideally, this learning objective can be formulated as a constrained optimization problem:

$$\min_{\theta} L_{\text{SL}}(\theta), \quad \text{s.t.} \quad \frac{\partial Q_{\psi}(s, \pi_{\theta}(s, \text{RTG}))}{\partial \text{RTG}} \geq 0, \quad (3)$$

where $L_{\text{SL}}(\theta)$ is the standard supervised learning loss ensuring the policy remains anchored to the offline dataset, while the constraint ensures the Q -function of the predicted action is monotonically increasing with respect to the input RTGs.

To preserve this monotonicity under the variations of RTGs,

directly calculating the gradient of critic Q can be time-consuming due to the backpropagation through the Q function and decision transformer. In order to improve the time efficiency, we consider the zero-th order estimation of the gradient following Salimans et al. (2017); Bernstein et al. (2018). In particular, for any small enough perturbation δ ,

$$\frac{\partial Q_\psi(s, \pi_\theta(s, \text{RTG}))}{\partial \text{RTG}} \approx \frac{Q_\psi(s, \hat{a}^\delta) - Q_\psi(s, \hat{a})}{\delta},$$

where $\hat{a}^\delta = \pi_\theta(s, \text{RTG} + \delta)$ and $\hat{a} = \pi_\theta(s, \text{RTG})$. To further mitigate the effect of the magnitude of δ and allow large perturbations δ , we convert (3) into the following objective:

$$\min_{\theta} L_{\text{SL}}(\theta), \text{ s.t. } \text{sgn}(\delta)(Q_\psi(s, \hat{a}^\delta) - Q_\psi(s, \hat{a})) \geq 0, \quad (4)$$

To this end, we solve (4) by introducing the Lagrange multiplier named as **alignment loss** L_{Align} . In particular, given an input sequence τ_t and its perturbed version τ_t^δ generated by adding a sequence-level noise $\delta \sim \mathcal{N}(0, \sigma_e^2)$, the alignment loss L_{Align} is defined by:

$$L_{\text{Align}} = \sum_{i=t-k+1}^t I_C \cdot |Q_\psi(s_i, \hat{a}_i^\delta) - Q_\psi^\perp(s_i, \hat{a}_i)|, \quad (5)$$

where $\hat{a}_i$ and $\hat{a}_i^\delta$ are the i -th predicted actions conditioned on τ_t and modified RTG sequence τ_t^δ defined by (2); $Q_\psi^\perp$ denotes the stop-gradient operator which is widely used in Q -based algorithms (Sutton & Barto, 2018) treating the original Q -value as a fixed reference to stabilize training; indicator function $\mathbb{I}_C$ detects the constraint violation in (4):

$$\mathbb{I}_C = \begin{cases} 1, & \text{if } \text{sgn}(\delta)(Q_\psi(s_i, \hat{a}_i^\delta) - Q_\psi^\perp(s_i, \hat{a}_i)) < 0 \\ 0, & \text{otherwise} \end{cases}.$$

For simplicity, let $Q_i^\delta = Q_\psi(s_i, \hat{a}_i^\delta)$ and $Q_i = Q_\psi^\perp(s_i, \hat{a}_i)$. The indicator activates when the RTG perturbation and the induced critic change have inconsistent directions, i.e., $\text{sgn}(\delta)(Q_i^\delta - Q_i) < 0$. This results in a directional ranking penalty that corrects violations of the RTG-value monotonicity while leaving already ordered pairs unchanged. In the context of offline RL, where Q -functions trained on static datasets are prone to scale bias and local inaccuracies (Kumar et al., 2019), this yields a *conservative alignment* objective by enforcing relative ordering rather than exploiting potentially inaccurate value magnitudes that could push the policy toward out-of-distribution actions (Fujimoto et al., 2019).

Together with the joint state-action prediction with loss $L_{\text{SL}}(\theta) = \sum_{i=t-k+1}^t \|s_i - \hat{s}_i\|^2 + \|a_i - \hat{a}_i\|^2$ following the decision transformer’s causal structure and input RTGs, this alignment objective encourages the model to internalize the RTG-action- Q mapping and regularize the policy to respect a partial RTG ordering. To this end, the overall training objective can be summarized as

$$\mathcal{L}_{\text{total}}(\theta) = L_{\text{SL}}(\theta) + \lambda_e L_{\text{Align}}(\theta), \quad (6)$$

where λ_e controls the relative weight of the alignment constraint. This formulation allows Q-ALIGN DT to learn a coherent family of policies that do not only imitate the dataset, but are also systematically steerable via RTG conditioning.

4.2. Training the Q Function with RTG Perturbation

In Q-ALIGN DT, a Q -function is required to compute the alignment loss. If the critic is either pretrained solely on the behavior policy or naively co-trained using actions generated by the policy conditioned on τ_t , it tends to merely reflect the offline data distribution and remains anchored to suboptimal returns. When the critic remains stationary, the alignment loss defined in (6) becomes self-limiting, as it penalizes the policy for attempting to exceed the RTGs present in the static dataset. In practice, this mismatch not only limits alignment in high-RTG regions, but can also degrade alignment across the entire RTG spectrum (see Table E.10).

To ensure the critic accurately reflects the policy’s behavior across the RTG spectrum, we train the critic to satisfy Bellman consistency with respect to the current policy’s response under RTG perturbations. Specifically, the critic Q_ψ is optimized to minimize:

$$L_q(\psi_{1,2}) = \sum_{i=t-k+1}^{t-1} \sum_{m=1}^2 (Q_{\psi_m}(s_i, a_i) - y'_i)^2, \quad (7)$$

with the target value y'_i being defined as:

$$y'_i = r_i + \gamma \min_{m=1,2} Q_{\psi'_m}^\perp(s_{i+1}, \hat{a}_{i+1}^{\prime, \Delta \text{RTG}}),$$

where $\hat{a}_{i+1}^{\prime, \Delta \text{RTG}}$ denotes the action predicted by the target policy $\pi_{\theta'}$ at the $(i+1)$ -th step conditioned on the perturbed trajectory sequence $\tau_t^{\Delta \text{RTG}}$ and $Q_{\psi'}$ is the target critic. The offset ΔRTG is a positive scalar that serves as a fixed trajectory-level perturbation. Conditioning on the RTG-perturbed trajectory $\tau_t^{\Delta \text{RTG}}$, the policy $\pi_{\theta'}$ induces a higher-return action support for the critic Q_ψ , encouraging the critic to reflect higher-return behaviors within the RTG-conditioned policy family. As shown in our analysis in Sec. 5.2, ΔRTG in the co-training mechanism effectively acts as a control parameter for the critic learning dynamics.

In particular, increasing ΔRTG biases the RTG-conditioned policy toward higher-return actions, generating more informative targets for critic learning. Through the alignment loss, these improved value estimates are fed back to the actor, enabling a positive actor-critic feedback loop that facilitates policy improvement while remaining grounded in the support of the offline data, rather than merely fitting static, suboptimal return averages.

Combined with the Q-ALIGN DT and the RTG-perturbation in co-training the Q function with the decision transformer, we present the algorithm as Algorithm 1 in Appendix A.

5. Theoretical Analysis

In this section, we analyze the alignment error of CSMs and the policy behavior of Q-ALIGN DT under data support constraints. We defer the detailed proof to Appendix B.

5.1. Why Q-ALIGN DT Reduces Alignment Error

Our analysis builds on the following finite sample analysis for DT presented in (Brandfonbrener et al., 2022):

Lemma 5.1 (Theorem 1 and Corollary 3, Brandfonbrener et al. 2022). *For a sample size N and a finite policy class Π , and an MDP with horizon H , let $\hat{\pi} \in \Pi$ be the empirical risk minimizer that minimizes the training loss. With probability at least $1 - \delta_p$, the total alignment error satisfies*

$$RTG_{igt} - \mathbb{E}_{\tau \sim \hat{\pi}_{RTG_{igt}}} [RTG_{real}] \leq \mathcal{O}\left(H^2 \frac{\log(|\Pi|/\delta_p)^{1/4}}{N^{1/4}}\right).$$

Crucially, the bound shows that alignment error scales with $(\log |\Pi|)^{1/4}$. In unconstrained sequence modeling, the policy class Π can be sufficiently large and therefore lead to significant alignment gap. In contrast, Q-ALIGN DT imposes an order-preserving bias that encourages $Q(s, \pi(s, g))$ to be monotonically increasing with respect to RTG g . As we will present in the following theorem, ensuring this monotonicity will effectively reduce the policy class Π .

Theorem 5.1 (Policy Space Reduction in Discrete Case). *Consider a discrete state and action space S, A , given a set of RTG values $G = \{g_1 < \dots < g_{|G|}\}$. Let Π_{free} be the class of all deterministic policies $\pi : S \times G \rightarrow A$. Assume that for each state s , the function $Q(s, \cdot) : A \rightarrow \mathcal{V}_s \subset \mathbb{R}$ induces an ordering over actions. A policy π is admissible in the constrained class Π_{mono} if it preserves this order across RTG levels. Then the log-complexity satisfies:*

$$\log |\Pi_{mono}| \leq |S||G| \log \frac{(|G| + |A| - 1)}{|G|} + |S||G|.$$

Furthermore, assuming each Q -value is shared by at most C actions and the resolution of the value space $|\mathcal{V}_s|$ is of the same order as the RTG resolution $|G|$ (i.e., $|\mathcal{V}_s| = \Theta(|G|)$), we have $\log |\Pi_{mono}| = \tilde{O}(|S||G|)$.

Compared to the unconstrained case where $\log |\Pi_{free}| = O(|S||G| \log |A|)$, Theorem 5.1 illustrates that Q-ALIGN DT utilizes the directional information of the Q function to effectively eliminate the $\log |A|$ dependence in the complexity term. This significant reduction in the hypothesis space provides a theoretical intuition for why Q-ALIGN DT can significantly diminish alignment errors observed in practice.

5.2. Policy Optimality under High-RTG Conditioning

As the alignment loss L_{Align} enables the partial ordering between the RTG and action quality, we would like to examine how the learned policy behaves when conditioned

on extremely large RTG values. This setting is particularly important in practice, as inputting sufficiently large RTGs is a standard evaluation protocol for CSMs.

To better understand this regime, the following theorem presents an asymptotic equivalent behavior of Q-ALIGN DT and QT (Hu et al., 2024), where QT explicitly optimizes for action-value maximization under fixed RTG.

Theorem 5.2 (Equivalence in High-RTG Regime). *For a fixed RTG R , let $\pi_\theta(s, R)$ be the policy learned by Q-ALIGN DT and $\pi_{QT}(s, R)$ be the policy that minimizes the following QT objective (Hu et al., 2024) using a fixed Q function:*

$$\mathcal{L}_{QT}(\pi, R) = \mathbb{E}_{(s,a) \sim \mathcal{D}} [\|\pi(s, R) - a\|^2 - \eta Q(s, \pi(s, R))].$$

Assuming the dataset $\mathcal{D}$ has full support over the action space, then as $R \rightarrow \infty$, Q-ALIGN DT recovers the behavior of an idealized QT that pursues maximum value:

$$\lim_{R \rightarrow \infty} \pi_\theta(s, R) = \lim_{R \rightarrow \infty} \pi_{QT}(s, R) = \arg \max_{a \in A} Q(s, a).$$

Remark 5.1. Theorem 5.2 highlights the robust extrapolation of Q-ALIGN DT under extreme conditioning. When prompted with an abnormally large RTG, the alignment objective prevents the policy from collapsing into erratic behavior or fragile interpolations. Instead, Q-ALIGN DT reverts to a value-maximizing mode similar to QT, utilizing the guidance of Q_ψ to select high-value actions.

Then the next theorem further characterizes the effectiveness of the RTG-perturbation techniques in learning the Q function used in Theorem 5.2. Especially, the following theorem shows different asymptotic behaviors under different choices of ΔRTG used in RTG-perturbation.

Theorem 5.3 (Impact of ΔRTG). *Let $\{Q_m, \pi_m\}_m$ be the sequences produced by Q-ALIGN DT. Consider the critic update using a perturbed conditioning signal $\tilde{R} = RTG + \Delta RTG$. Then the following statements hold:*

(No Perturbation). *If $\Delta RTG=0$, the updates remain within the behavior policy’s action support, leading to convergence of $\{Q_m\}_m$ to a value function close to Q_β , corresponding to conservative evaluation within the behavior support.*

(Large Perturbation). *If ΔRTG is sufficiently large such that $\tilde{R}$ exceeds all returns in $\mathcal{D}$, then $\{Q_m\}_m$ converges to the optimal action-value function Q^* and the induced policies $\{\pi_m(s, \tilde{R})\}_m$ converge to the corresponding optimal policy π^* , both restricted to the support of $\mathcal{D}$.*

Remark 5.2. Theorem 5.3 characterizes the effect of ΔRTG on the behavior of the critic update: at the extremes when ΔRTG approaches 0, RTG-perturbation recovers a SARSA-style policy evaluation, while a sufficiently large ΔRTG approximates standard Q -learning (Sutton & Barto,

2018). For intermediate values, the update interpolates between these behaviors, conceptually similar to the expectile parameter in IQL (Kostrikov et al., 2022). This implies that ΔRTG can be used to guide the learning of Q by providing a more structured and higher-quality action space. In practice, while a larger ΔRTG encourages the discovery of optimal policies, it can also increase optimization difficulty and potential instability; we therefore treat ΔRTG as a tunable hyperparameter to balance policy improvement and training stability. We further explore the impact of different choices of ΔRTG in Sec. 6.4.

6. Experiments

We evaluate the Q-ALIGN DT mainly on D4RL benchmark (Fu et al., 2021) including the Gym (Hopper, HalfCheetah, Walker2d) and AntMaze (umaze, medium) tasks.

6.1. Implementation Details

We compare Q-ALIGN DT with a diverse set of baselines, including both value-based methods and CSM methods. For value-based approaches, we consider IQL (Kostrikov et al., 2022), TD3+BC (Fujimoto & Gu, 2021), and CQL (Kumar et al., 2020). For CSM-based methods, we include DT (Chen et al., 2021), DC (Kim et al., 2024a), RVS (Emmons et al., 2022), CGDT (Wang et al., 2024), LSDT (Wang et al., 2025), DM (Zheng et al., 2025), RADT (Tanaka et al., 2025), QT (Hu et al., 2024), and QCS (Kim et al., 2024b).

For training stability, we pretrain the Q -function on the offline dataset. To decouple the performance of RTG-to-behavior alignment from the effects of critic pretraining, we employ a standard Double Q -learning update (Hasselt et al., 2016) across most environments, with the exception of antmaze, where IQL (Kostrikov et al., 2022) is adopted due to its superior performance in sparse-reward tasks (see Appendix D.2 for details).

Motivated by prior observations that vanilla attention underutilizes RTG tokens (Kim et al., 2024a; Tanaka et al., 2025), we empirically introduce a lightweight convolutional projection for attention inputs. As shown in Table 4, this does not affect peak performance but improves alignment behavior; architectural details and ablations are provided in Appendix D.1.

6.2. Metric

We follow standard offline RL evaluation and report D4RL normalized scores averaged over five seeds, with 50 rollouts per seed (100 for AntMaze).

To evaluate *alignment* error, we measure how accurately a policy tracks a prescribed RTG target. For each environment, we sweep target RTGs from the minimum to maxi-

mum D4RL return in increments of 100, execute 30 rollouts per target, and record the resulting normalized scores. Let $\text{score}_{\text{tgt},j}$ and $\text{score}_{\text{real},j}$ denote the j -th target and achieved scores. For n target RTGs, we evaluate the alignment error by the root mean squared deviation

$$M = \sqrt{\frac{1}{n} \sum_{j=1}^n (\text{score}_{\text{real},j} - \text{score}_{\text{tgt},j})^2}, \quad (8)$$

where lower M indicates better RTG-to-behavior alignment.

6.3. Main Results

We first present the performance of Q-ALIGN DT on Gym domains and AntMaze tasks in Tables 1 and 2. The results show that Q-ALIGN DT consistently achieves state-of-the-art performance, and in many cases surpasses existing methods across the datasets. This demonstrates that, although our method is primarily motivated by alignment, it can attain high scores when conditioned on high RTG, as is typical for CSMs.

We also report alignment metrics in Table 3. Q-ALIGN DT achieves strong alignment across the evaluated Gym domains, validating the effectiveness of our proposed approach. Notably, in HalfCheetah, where many CSMs tend to produce similar behaviors for different RTGs, our method reliably aligns with the target RTG.

6.4. Ablation Study

In this section, we present key ablation studies for Q-ALIGN DT. Additional experiments, including hyperparameter variations, alternative indicator functions, architectural components, and training procedures, are reported in Appendix E.1.

Selection of ΔRTG in *RTG-perturbation*. To evaluate the impact of different ΔRTG values, we conduct experiments on halfcheetah-medium with normalized $\Delta\text{RTG} \in \{0, 1, 3, 5, 10\}$ (i.e., actual RTG normalized by 1000). We report both the D4RL score and the alignment error in Figure 2. Consistent with our analysis, increasing ΔRTG generally improves overall performance, while the alignment error increases mildly.

Interestingly, alignment error increases as the RTG offset approaches zero, with most degradation occurring in the low-RTG regime. This effect is closely related to early-timestep collapse and is exacerbated when the Q -function is fixed after pretraining (Appendix E.5). We hypothesize that this occurs because the critic remains close to Q_β . In low-RTG regions, this leads to weak and noisy guidance due to the highly multimodal nature of low-return behaviors and their limited value separation. Consequently, RTG perturbations are important not only for high-RTG performance, but also for maintaining reliable alignment across the RTG spectrum.

Table 1. Performance (D4RL normalized score $\uparrow$) of Q-ALIGN DT and other state-of-the-art baselines on Gym domains. Results are averaged over five random seeds, and we report the mean $\pm$ standard error. Boldface numbers denote the highest or comparable scores among the algorithms.

Dataset	IQL	TD3+BC	DT	CGDT	LSDT	DC	DM	RADT	QT	QCS	Q-ALIGN DT
halfcheetah-medium-replay	44.1	44.6	36.6	40.4	42.9	41.3	39.6	41.3	48.9	54.1	57.1 $\pm$ 0.74
hopper-medium-replay	92.1	60.9	82.7	93.4	93.9	94.2	95.4	95.7	102	100.4	102.2 $\pm$ 0.64
walker2d-medium-replay	73.7	81.8	79.4	78.1	74.7	76.6	85.5	75.9	98.5	94.1	101.3 $\pm$ 0.73
halfcheetah-medium	47.4	48.3	42.6	43	43.6	43	43.5	—	51.4	59	65.3 $\pm$ 0.63
hopper-medium	63.8	59.3	67.6	96.9	87.2	92.5	98.1	—	96.9	96.4	102.1 $\pm$ 0.74
walker2d-medium	79.9	83.7	74	79.1	81	79.2	83.8	—	88.8	88.2	94.7 $\pm$ 0.67
halfcheetah-medium-expert	86.7	90.7	86.8	93.6	93.2	93	93.9	93.1	96.1	93.3	98.8 $\pm$ 0.68
hopper-medium-expert	91.5	98	107.6	107.6	111.7	110.4	111.8	110.4	113.4	110.2	114.0 $\pm$ 0.18
walker2d-medium-expert	109.6	110.1	108.1	109.3	109.8	109.6	112.7	109.7	112.6	116.6	121.4 $\pm$ 0.52
Sum	688.8	677.4	685.4	741.4	738	739.8	764.3	—	808.6	812.3	856.9

Table 2. Performance (D4RL normalized score $\uparrow$) of Q-ALIGN DT and baseline methods on AntMaze. Results averaged over five seeds. Boldface numbers denote the highest or comparable scores among the algorithms.

Dataset	IQL	TD3+BC	CQL	DT	CGDT	RVS	DC	QT	QCS	Q-ALIGN DT
antmaze-umaze	87.5	78.6	74	65.6	71	64.4	85.0	90.7	92.7	96.4 $\pm$ 1.12
antmaze-umaze-diverse	62.2	71.4	84	51.2	71	70.1	78.5	83.7	72.3	73.2 $\pm$ 6.44
antmaze-medium-play	71.2	10.6	61.2	4.3	—	4.5	33.2	78.6	81.6	85.6 $\pm$ 4.74
antmaze-medium-diverse	70	3	53.7	1.2	—	7.7	27.5	62.3	79.5	76.2 $\pm$ 6.62

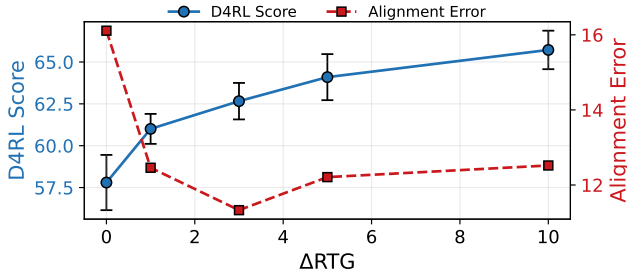


Figure 2. Effect of normalized RTG offset Δ RTG (actual RTG divided by 1000) on policy performance (D4RL score) and alignment error in *halfcheetah-medium*.

On the other hand, larger RTG offsets target higher-return behaviors but also increase distributional shift, potentially destabilizing Q -function learning. To mitigate this, we restrict Δ RTG to a moderate range; detailed values and experimental settings are reported in Appendix D.5.

Effect of the Convolution Layer. We ablate the 1D convolution layer in our architecture by reverting the model to the standard Decision Transformer. As shown in Table 4, while the convolution layer has a marginal impact on the best achievable performance, it substantially reduces the alignment error. A possible explanation is that the convolution operation enables the attention mechanism to better extract and propagate information from the RTG token, thereby providing more informative alignment signals.

7. Behavioral Analysis and Generalization

While D4RL rewards are usually scalar, these rewards often aggregate multiple components. For instance, *halfcheetah* uses the difference between the forward speed *forward_reward* and *ctrl_cost* as the reward. Therefore, it’s natural to ask whether the model can meaningfully adjust its behavior conditioned on different target RTGs even when trained only on scalar returns.

To answer this, we evaluate rollouts across a range of target RTGs. Higher RTGs consistently induce faster locomotion, while lower RTGs lead to more cautious movement (Figure 3). A finer-grained analysis of instantaneous velocity along representative trajectories (Figure 4) shows that the agent adaptively adjusts gait and speed rather than following a single policy to satisfy the specified reward constraints. These results suggest that Q-ALIGN DT has captured a semantically meaningful mapping from RTG tokens to the underlying action space. Additional ant experiments are provided in Appendix E.3 with similar observations.

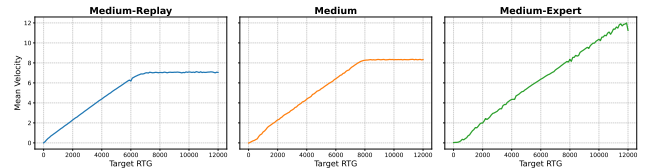


Figure 3. Mean forward velocity of the agent conditioned on varying target RTGs across different datasets on *halfcheetah*.

Table 3. Alignment performance (Root Mean Squared Error $\downarrow$) of Q-ALIGN DT and other state-of-the-art baselines on Gym domains. Results are averaged over five random seeds.

Dataset	DC	RADT	DT	QT	QCS	Q-ALIGN DT
walker2d-medium-replay	14.05 $\pm$ 0.11	14.27 $\pm$ 2.09	18.67 $\pm$ 1.08	35.93 $\pm$ 1.9	20.55 $\pm$ 3.26	6.39 $\pm$ 0.51
hopper-medium-replay	12.65 $\pm$ 0.3	6.49 $\pm$ 0.16	11.38 $\pm$ 1.31	39.69 $\pm$ 7.4	27.49 $\pm$ 3.09	10.1 $\pm$ 1.2
halfcheetah-medium-replay	28.67 $\pm$ 0.34	50.15 $\pm$ 1.45	30.93 $\pm$ 0.59	28.25 $\pm$ 0.35	28.12 $\pm$ 0.52	17.8 $\pm$ 0.53
walker2d-medium	16.72 $\pm$ 3.77	18.68 $\pm$ 4.46	30.69 $\pm$ 1.99	38.25 $\pm$ 7.99	40.85 $\pm$ 2.95	6.25 $\pm$ 0.43
hopper-medium	16.03 $\pm$ 2.45	14.99 $\pm$ 0.74	16.08 $\pm$ 0.38	31.42 $\pm$ 1.92	32.19 $\pm$ 0.51	8.39 $\pm$ 2.3
halfcheetah-medium	28.89 $\pm$ 0.1	25.23 $\pm$ 1.21	29.22 $\pm$ 0.28	28.07 $\pm$ 0.32	28.88 $\pm$ 0.1	12.36 $\pm$ 0.7
walker2d-medium-expert	13.62 $\pm$ 1.59	10.15 $\pm$ 0.69	18.88 $\pm$ 1.07	63.71 $\pm$ 1.09	48.37 $\pm$ 0.44	6.84 $\pm$ 0.83
hopper-medium-expert	12.22 $\pm$ 0.8	8.18 $\pm$ 0.81	16.92 $\pm$ 1.89	21.39 $\pm$ 0.1	24.91 $\pm$ 0.41	11.7 $\pm$ 2.05
halfcheetah-medium-expert	17.88 $\pm$ 0.17	17.55 $\pm$ 0.1	19.79 $\pm$ 1.74	21.24 $\pm$ 0.96	43.95 $\pm$ 1.1	1.24 $\pm$ 0.19
Mean	17.86	18.58	21.39	34.22	32.81	9.01

Table 4. Effect of the convolution layer. Align. RMSE ($\downarrow$) denotes the alignment error, Perf. ($\uparrow$) denotes the overall performance (D4RL normalized score). Results averaged over three seeds.

Dataset	Align. RMSE $\downarrow$		Perf. $\uparrow$	
	Ours	w/o Conv.	Ours	w/o Conv.
Medium-Replay	11.31	13.37	86.45	86.88
Medium	9.19	14.22	87.70	87.21
Medium-Expert	6.80	10.70	111.37	110.94

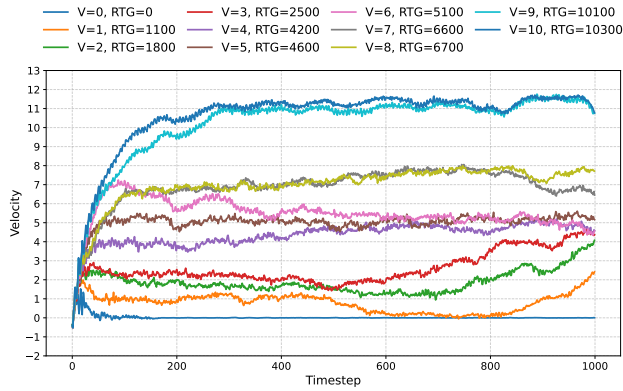


Figure 4. Agent velocity over timesteps for different target RTGs on halfcheetah-medium-expert, averaged over 30 runs.

Zero-shot transfer to Meta-RL tasks. With this observation, we further challenge the model’s alignment robustness through a zero-shot transfer to *halfcheetah-vel* (Mitchell et al., 2021). We compare our approach against two distinct categories of baselines: (i) **In-domain Meta-RL methods**, including Prompt-DT (Xu et al., 2022), MACAW (Mitchell et al., 2021), and PEARL (Rakelly et al., 2019), which are trained on the *halfcheetah-vel* task family across multiple tasks; and (ii) **Cross-domain CSMs**, which, like Q-ALIGN DT, are trained exclusively on the *halfcheetah-medium-expert* dataset and generalize to the velocity-tracking task directly.

Table 5. **Performance on HalfCheetah-vel.** Meta-RL methods are trained on *HalfCheetah-vel*, while offline RL methods are trained on *HalfCheetah-medium-expert* and evaluated via zero-shot transfer. For Prompt-DT, $K^*=5$ follows the original setting with 5-step expert prompts used at both training and inference, while $K^*=0$ removes prompts entirely to enable a fair comparison with prompt-free CSMs.

Method	Average Episode Return
<i>Trained on HalfCheetah-vel</i>	
MACAW (Mitchell et al., 2021)	-121.6
PEARL (Rakelly et al., 2019)	-273.6
Prompt-DT (Xu et al., 2022) ($K^*=0$)	-188.4
Prompt-DT ($K^*=5$)	-38.6
<i>Trained on HalfCheetah-medium-expert</i>	
RADT (Tanaka et al., 2025)	-754.7
DC (Kim et al., 2024a)	-756.7
DT (Chen et al., 2021)	-764.6
QCS (Kim et al., 2024b)	-1298.7
QT (Hu et al., 2024)	-824.6
Q-ALIGN DT	-142.3

To account for differences in reward formulation, observation dimensions, and episode lengths, we apply a unified adjustment protocol to all cross-domain CSMs (details in Appendix C). Specifically, for each target velocity, the agent’s behavior is controlled solely by the input RTG, which is determined using simple linear interpolation.

Surprisingly, we found that Q-ALIGN DT exhibits a strong zero-shot transferability compared to other CSMs and achieves competitive performance even against in-domain Meta-RL baselines as shown in Table 5. This demonstrates its superior ability to represent and generalize multiple RTG-conditioned policies across environments with significant discrepancies. Furthermore, we evaluate the model on higher target velocities that exceed the original range of the *halfcheetah-vel* benchmark in Table E.15. These results highlight the controllability of Q-ALIGN DT and its ability to generalize to a wide range of target velocities.

8. Conclusion

In this work, we introduce Q-ALIGN DT, a framework designed to align conditioned sequence models with different return-to-go (RTG) targets. We analyze the underlying mechanisms of Q-ALIGN DT and show that actions from alignment-consistent CSMs can provide informative signals for Q -function learning. Extensive experiments demonstrate that our approach learns a coherent family of policies and achieves state-of-the-art performance in both return maximization and RTG alignment across a wide range of environments.

Acknowledgements

We thank the anonymous reviewers for their helpful comments. This research was supported by WZ’s startup funding provided by the School of Data Science and Society at UNC Chapel Hill.

Impact Statement

This paper presents work whose primary goal is to advance the field of machine learning, specifically in offline reinforcement learning and conditioned sequence models. Our contributions focus on improving the alignment and controllability of learned policies using fixed datasets, without introducing new data sources or deployment mechanisms.

The techniques studied in this work are general algorithmic methods and are evaluated exclusively in standard simulated benchmark environments. As such, we do not anticipate direct negative societal impacts arising uniquely from this research beyond those already well established for reinforcement learning methods in general. Potential applications of improved policy alignment include enabling safer and more controllable decision-making systems, subject to appropriate domain-specific considerations.

Limitations and Future Work

This paper studies the RTG mismatch issue in return-conditioned offline RL and proposes Q-guided alignment as a practical mitigation. Our experiments focus on standard Gym and AntMaze benchmarks, and extending the evaluation to broader domains such as visual control and robotic manipulation is an important direction for future work. Our theoretical analysis uses simplifying assumptions to isolate the effect of RTG–behavior alignment; extending the analysis to more general settings remains open. Finally, while Q-ALIGN DT shows empirical benefits in sparse-reward environments such as AntMaze, its stable performance in these settings currently relies on additional initialization or preprocessing for obtaining a reliable critic signal. Developing a more robust sparse-reward variant is an interesting

direction for future work.

References

- Agarwal, R., Schuurmans, D., and Norouzi, M. An optimistic perspective on offline reinforcement learning. In III, H. D. and Singh, A. (eds.), *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pp. 104–114. PMLR, 13–18 Jul 2020.
- Beck, J., Vuorio, R., Zheran Liu, E., Xiong, Z., Zintgraf, L., Finn, C., and Whiteson, S. A tutorial on meta-reinforcement learning. *Foundations and Trends in Machine Learning*, 18(2–3):224–384, April 2025. ISSN 1935-8245. doi: 10.1561/22000000080.
- Bernstein, J., Wang, Y.-X., Azizzadenesheli, K., and Anandkumar, A. signSGD: Compressed optimisation for non-convex problems. In Dy, J. and Krause, A. (eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 560–569. PMLR, 10–15 Jul 2018.
- Brandfonbrener, D., Bietti, A., Buckman, J., Larocche, R., and Bruna, J. When does return-conditioned supervised learning work for offline reinforcement learning? In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners, 2020.
- Chebotar, Y., Vuong, Q., Hausman, K., Xia, F., Lu, Y., Irpan, A., Kumar, A., Yu, T., Herzog, A., Pertsch, K., Gopalakrishnan, K., Ibarz, J., Nachum, O., Sontakke, S. A., Salazar, G., Tran, H. T., Peralta, J., Tan, C., Manjunath, D., Singh, J., Zitkovich, B., Jackson, T., Rao, K., Finn, C., and Levine, S. Q-transformer: Scalable offline reinforcement learning via autoregressive q-functions. In Tan, J., Toussaint, M., and Darvish, K. (eds.), *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pp. 3909–3928. PMLR, 06–09 Nov 2023.
- Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., and Mordatch, I. Decision transformer: Reinforcement learning via sequence modeling. In Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, 2021.

- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423.
- Dorfman, R., Shenfeld, I., and Tamar, A. Offline meta reinforcement learning – identifiability challenges and effective data collection strategies. In Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 4607–4618. Curran Associates, Inc., 2021.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021a.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. An image is worth 16x16 words: Transformers for image recognition at scale, 2021b.
- Duan, Y., Schulman, J., Chen, X., Bartlett, P. L., Sutskever, I., and Abbeel, P. RL^2 : Fast reinforcement learning via slow reinforcement learning, 2016.
- Emmons, S., Eysenbach, B., Kostrikov, I., and Levine, S. Rvs: What is essential for offline RL via supervised learning? In *International Conference on Learning Representations*, 2022.
- Ernst, D., Geurts, P., and Wehenkel, L. Tree-based batch mode reinforcement learning. *Journal of Machine Learning Research*, 6(18):503–556, 2005.
- Finn, C., Abbeel, P., and Levine, S. Model-agnostic meta-learning for fast adaptation of deep networks, 2017.
- Fu, J., Kumar, A., Nachum, O., Tucker, G., and Levine, S. D4{rl}: Datasets for deep data-driven reinforcement learning, 2021.
- Fujimoto, S. and Gu, S. A minimalist approach to offline reinforcement learning. In Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, 2021.
- Fujimoto, S., Meger, D., and Precup, D. Off-policy deep reinforcement learning without exploration. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2052–2062. PMLR, 09–15 Jun 2019.
- Furuta, H., Matsuo, Y., and Gu, S. S. Generalized decision transformer for offline hindsight information matching. In *International Conference on Learning Representations*, 2022.
- Grigsby, J., Fan, L., and Zhu, Y. AMAGO: Scalable in-context reinforcement learning for adaptive agents. In *The Twelfth International Conference on Learning Representations*, 2024.
- Hasselt, H. v., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, AAAI’16, pp. 2094–2100. AAAI Press, 2016.
- He, K., Chen, X., Xie, S., Li, Y., Dollár, P., and Girshick, R. Masked autoencoders are scalable vision learners, 2021.
- Hendrycks, D. and Gimpel, K. Gaussian error linear units (gelus). *arXiv: Learning*, 2016.
- Hu, S., Fan, Z., Huang, C., Shen, L., Zhang, Y., Wang, Y., and Tao, D. Q-value regularized transformer for offline reinforcement learning. In *Forty-first International Conference on Machine Learning*, 2024.
- Janner, M., Li, Q., and Levine, S. Offline reinforcement learning as one big sequence modeling problem. In Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, 2021.
- Jaques, N., Ghandeharioun, A., Shen, J. H., Ferguson, C., Lapedriza, A., Jones, N., Gu, S., and Picard, R. Way off-policy batch deep reinforcement learning of implicit human preferences in dialog, 2019.
- Kim, J., Lee, S., Kim, W., and Sung, Y. Decision convformer: Local filtering in metaformer is sufficient for decision making. In *The Twelfth International Conference on Learning Representations*, 2024a.
- Kim, J., Lee, S., Kim, W., and Sung, Y. Adaptive \$q\$-aid for conditional supervised learning in offline reinforcement learning. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *CoRR*, abs/1412.6980, 2014.

- Kostrikov, I., Nair, A., and Levine, S. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations*, 2022.
- Kumar, A., Fu, J., Tucker, G., and Levine, S. Stabilizing off-policy q-learning via bootstrapping error reduction. In *Neural Information Processing Systems*, 2019.
- Kumar, A., Zhou, A., Tucker, G., and Levine, S. Conservative q-learning for offline reinforcement learning. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1179–1191. Curran Associates, Inc., 2020.
- Laskin, M., Wang, L., Oh, J., Parisotto, E., Spencer, S., Steigerwald, R., Strouse, D., Hansen, S. S., Filos, A., Brooks, E., maxime gazeau, Sahni, H., Singh, S., and Mnih, V. In-context reinforcement learning with algorithm distillation. In *The Eleventh International Conference on Learning Representations*, 2023.
- Lee, J., Xie, A., Pacchiano, A., Chandak, Y., Finn, C., Nachum, O., and Brunskill, E. Supervised pretraining can learn in-context reinforcement learning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- Levine, S., Kumar, A., Tucker, G., and Fu, J. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020.
- Lin, L., Bai, Y., and Mei, S. Transformers as decision makers: Provable in-context reinforcement learning via supervised pretraining. In *The Twelfth International Conference on Learning Representations*, 2024.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. Roberta: A robustly optimized bert pretraining approach. *ArXiv*, abs/1907.11692, 2019.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin transformer: Hierarchical vision transformer using shifted windows, 2021.
- Mitchell, E., Rafailov, R., Peng, X. B., Levine, S., and Finn, C. Offline meta-reinforcement learning with advantage weighting. In Meila, M. and Zhang, T. (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 7780–7791. PMLR, 18–24 Jul 2021.
- Rakelly, K., Zhou, A., Finn, C., Levine, S., and Quillen, D. Efficient off-policy meta-reinforcement learning via probabilistic context variables. In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 5331–5340. PMLR, 09–15 Jun 2019.
- Salimans, T., Ho, J., Chen, X., Sidor, S., and Sutskever, I. Evolution strategies as a scalable alternative to reinforcement learning, 2017.
- Siegel, N. Y., Springenberg, J. T., Berkenkamp, F., Abdolmaleki, A., Neunert, M., Lampe, T., Hafner, R., Heess, N., and Riedmiller, M. Keep doing what worked: Behavioral modelling priors for offline reinforcement learning, 2020.
- Sutton, R. S. and Barto, A. G. *Reinforcement Learning: An Introduction*. A Bradford Book, Cambridge, MA, USA, 2018. ISBN 0262039249.
- Tanaka, T., Abe, K., Ariu, K., Morimura, T., and Simo-Serra, E. Return-aligned decision transformer. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS’17, pp. 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.
- Wang, J., Karanasou, P., Wei, P., Gatti, E., Plasencia, D. M., and Kanoulas, D. Long-short decision transformer: Bridging global and local dependencies for generalized decision-making. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Wang, Y., Yang, C., Wen, Y., Liu, Y., and Qiao, Y. Critic-guided decision transformer for offline reinforcement learning. In *AAAI*, pp. 15706–15714, 2024.
- Wang, Z., Novikov, A., Zolna, K., Merel, J. S., Springenberg, J. T., Reed, S. E., Shahriari, B., Siegel, N., Gulcehre, C., Heess, N., and de Freitas, N. Critic regularized regression. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 7768–7778. Curran Associates, Inc., 2020.
- Wu, Y., Tucker, G., and Nachum, O. Behavior regularized offline reinforcement learning, 2019.
- Wu, Y.-H., Wang, X., and Hamaya, M. Elastic decision transformer. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- Xu, M., Shen, Y., Zhang, S., Lu, Y., Zhao, D., Tenenbaum, B. J., and Gan, C. Prompting decision transformer for few-shot policy generalization. In *Thirty-ninth International Conference on Machine Learning*, 2022.

Yamagata, T., Khalil, A., and Santos-Rodriguez, R. Q-learning decision transformer: Leveraging dynamic programming for conditional sequence modelling in offline RL. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 38989–39007. PMLR, 23–29 Jul 2023.

Yan, K., Schwing, A. G., and Wang, Y.-X. Reinforcement learning gradients as vitamin for online finetuning decision transformers. In *Neural Information Processing Systems (NeurIPS)*, 2024.

Zheng, H., Shen, L., Luo, Y., Ye, D., Du, B., Shen, J., and Tao, D. Decision mixer: Integrating long-term and local dependencies via dynamic token selection for decision-making. In *Forty-second International Conference on Machine Learning*, 2025.

A. Algorithm

In this section, we provide the formal pseudocode for Q-ALIGN DT. The training procedure is designed to ensure that the policy π_θ not only clones the behavioral data but also adheres to directional value consistency through the alignment loss.

Algorithm 1 Training Procedure

Input: Dataset $\mathcal{D}$, sequence horizon k , training epochs N , batch size B , RTG offset ΔRTG , noise variance σ_e^2 , target update rate α , pretrained critic networks Q_{ψ_1}, Q_{ψ_2} , target critic networks $Q_{\psi'_1}, Q_{\psi'_2}$

Output: Trained actor parameters θ

```

1: Initialize actor  $\pi_\theta$  and target actor  $\pi_{\theta'}$ 
2: for epoch = 1 to  $N$  do
3:   Sample batch of trajectories  $\tau \sim \mathcal{D}$ 
4:   Sample sub-trajectory  $\tau_t$  of length  $k$  with random timestep  $t$ 
   // Critic update
5:   Sample target actions  $\hat{a}'^{\Delta\text{RTG}} \sim \pi_{\theta'}(\tau_t^{\Delta\text{RTG}})$ 
6:   Update critic networks using (7)
   // Actor update
7:   Predict actions  $\hat{a} \sim \pi_\theta(\tau_t)$ 
8:   Sample noise  $\delta \sim \mathcal{N}(0, \sigma_e^2)$ 
9:   Predict noisy actions  $\hat{a}^\delta \sim \pi_\theta(\tau_t^\delta)$ 
10:  Update actor using (6)
11:  Update target networks (actor and critics) with rate  $\alpha$ 
12: end for
13: Return  $\theta$ 
    
```

B. Mathematical Details

B.1. Proof of Theorem 5.1

In order to prove Theorem 5.1, we first give a formal statement and analysis of results from Brandfonbrener et al. (2022).

Theorem B.1. Consider an MDP, a behavior policy β , and a conditioning function f that is consistent with the reward dynamics, i.e.,

$$f(s) = f(s') + r(s, a)$$

for any transition from s to s' under action a . Let

$$g(\tau) = \sum_{i=0}^H r_i$$

denote the cumulative reward of a trajectory τ . Assume:

1. **Return Coverage:** the policy β covers all initial states s_1 , i.e.,

$$P(g(\tau) = (f(s_1) \mid s_1) \geq \alpha_f, \quad \forall s_1.$$

2. **Near Determinism:** the environment satisfies

$$P(r \neq R(s, a) \mid s, a) \leq \epsilon, \quad P(s' \neq T(s, a) \mid s, a) \leq \epsilon$$

for some reward function r and transition function T .

Then the expected return of the conditioned policy trained on infinite data π_f satisfies

$$\mathbb{E}_{s_1}[f(s_1)] - \mathbb{E}_{\tau \sim \pi_f}[g(\tau)] \leq \epsilon \left(\frac{1}{\alpha_f} + 2 \right) H^2,$$

where H is the horizon.

The result follows directly from Theorem 1 of Brandfonbrener et al. (2022). $\square$

With Theorem B.1, we can analyze why previous CSMs struggle to align with target RTG values that deviate from the training distribution. Following Yan et al. (2024), we consider the case where $f(s_1) = \text{RTG}_{\text{tgt}}$. According to Theorem B.1, the alignment error is bounded by:

$$\text{RTG}_{\text{tgt}} - \mathbb{E}_{\tau \sim \pi(\cdot|s, \text{RTG}_{\text{tgt}})}[\text{RTG}_{\text{real}}] \leq \epsilon \left(\frac{1}{\alpha_f} + 2 \right) H^2,$$

While Theorem B.1 provides an analysis of DT's behavior in the infinite-data regime, we are more interested in the finite-data setting. In particular, we have:

Theorem B.2. *Under the assumptions of Theorem B.1, let the training sample size be N , and further assume that*

- $\frac{P_{\pi_f}(s)}{P_{\pi_\beta}(s)} \leq C_f$ for all s .
- The policy class Π is finite.
- $|\log \pi(a|s, g) - \log \pi(a'|s', g')| < c$ for all (a, s, g, a', s', g') and all $\pi \in \Pi$.
- The approximation error of Π is bounded by ϵ_{error} .

Then, with probability at least $1 - \delta_p$,

$$\mathbb{E}_{\tau \sim \pi(\cdot|s, \text{RTG}_{\text{tgt}})}[\text{RTG}_{\text{real}}] - \mathbb{E}_{\tau \sim \hat{\pi}(\cdot|s, \text{RTG}_{\text{tgt}})}[\text{RTG}_{\text{real}}] \leq O\left(\frac{C_f}{\alpha_f} H^2 \left(\sqrt{c} \left(\frac{\log |\Pi| / \delta_p}{N}\right)^{1/4}\right) + \sqrt{\epsilon_{\text{error}}}\right).$$

where $\hat{\pi} \in \Pi$ is the policy that minimizes the KL divergence on the training dataset.

The result follows directly from Corollary 3 in Brandfonbrener et al. (2022). $\square$

Combining Theorem B.1 and Theorem B.2, we conclude that with probability at least $1 - \delta_p$, the total alignment error of the learned policy $\hat{\pi}$ scales as

$$\text{RTG}_{\text{tgt}} - \mathbb{E}_{\tau \sim \hat{\pi}(\cdot|s, \text{RTG}_{\text{tgt}})}[\text{RTG}_{\text{real}}] \leq O\left(\epsilon \frac{1}{\alpha_f} H^2 + \frac{C_f}{\alpha_f} H^2 \left(\sqrt{c} \left(\frac{\log |\Pi| / \delta_p}{N}\right)^{1/4}\right) + \sqrt{\epsilon_{\text{error}}}\right). \quad (9)$$

By imposing a monotonicity constraint on the Q -value of the policy output with respect to the given RTG, Q-ALIGN DT effectively restricts the policy space Π . We then give the proof of Theorem 5.1:

Proof of Theorem 5.1. In the unconstrained case, each state-RTG pair (s, g) can be independently mapped to any of $|A|$ actions. With $|S|$ states and K RTG levels, the total number of policies is:

$$|\Pi_{\text{free}}| = |A|^{|S||G|}, \quad \log |\Pi_{\text{free}}| = |S||G| \log |A|.$$

In the constrained case Π_{mono} , for a fixed state s , the sequence $(\pi(s, g_1), \dots, \pi(s, g_{|G|}))$ must be non-decreasing with respect to $\preceq_s$ induced by the scalar-valued $Q(s, \cdot)$. In the presence of ties in $Q(s, \cdot)$, we assume an arbitrary but fixed tie-breaking rule, which induces a total order $\preceq_s$ compatible with $Q(s, \cdot)$. Treating each state independently, the number of monotone sequences of length K over the set of actions A is given by the multiset coefficient:

$$|\Pi_{\text{mono}, s}| = \binom{|G| + |A| - 1}{|G|}.$$

Across all states:

$$|\Pi_{\text{mono}}| = \prod_{s \in \mathcal{S}} |\Pi_{\text{mono},s}|, \quad \log |\Pi_{\text{mono}}| = \sum_{s \in \mathcal{S}} \log \binom{|G| + |A| - 1}{|G|}.$$

Using $\binom{n}{k} \leq (en/k)^k$, we have

$$\log |\Pi_{\text{mono}}| \leq |S||G| \log \frac{e(|G| + |A| - 1)}{|G|}.$$

If we further consider the regime where the resolution of the score space $|\mathcal{V}_s|$ is comparable to the RTG discretization $|G|$ (i.e., $|\mathcal{V}_s| = \Theta(|G|)$), then, since each score value can be shared by at most a constant number C of actions, we have $|A| \leq C|\mathcal{V}_s| = \Theta(|G|)$. Therefore,

$$\frac{|G| + |A| - 1}{|G|} = O(1).$$

Consequently, the resulting log-complexity bound satisfies

$$\log |\Pi_{\text{mono}}| = O(|S||G|),$$

eliminating any explicit logarithmic dependence on $|A|$.

Remark B.1 (On the Comparability of RTG and Score Resolution). The assumption $|\mathcal{V}_s| = \Theta(|G|)$ is reasonable in our setting. RTG corresponds to undiscounted returns ($\gamma = 1$), while the critic Q_ψ provides a discounted estimate of action quality; both therefore encode return-based preferences at comparable resolution. Moreover, RTG perturbations are sampled with non-negligible variance (see Table D.8), encouraging consistent policy responses over a broad RTG range rather than infinitesimal changes.

□

B.2. Proof of Theorem 5.2

We consider an MDP

$$M = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

with horizon H , where $P(s' | s, a)$ denotes the transition probability. Let R^* be a sufficiently large total reward, larger than the return of any trajectory $\tau = (s_0, a_0, r_0, \dots, s_H, a_H, r_H)$ in the environment.

Our proof strategy is to decompose Theorem 5.2 into two parts, formally stated as two lemmas. These lemmas show that both QT and Q-ALIGN DT's objectives converge to the same greedy action-selection criterion when the target RTG is sufficiently large.

Lemma B.1. *Let Q be a fixed critic defined for all (s, a) pairs in the dataset, and let π^{Align} denote the policy that minimizes the alignment actor loss in (6). Assume the dataset fully covers all state-action pairs, i.e.,*

$$p_{\mathcal{D}}(a | s) > 0, \quad \forall s \in \mathcal{S}, \forall a \in \mathcal{A},$$

where $p_{\mathcal{D}}(a | s)$ denotes the empirical probability of observing action a at state s in the dataset. Then, for the maximal target return R^* , the policy $\pi_{R^*}^{\text{Align}}$ outputs an action

$$a^* = \arg \max_{a \in \mathcal{A}_{\text{data}}(s)} Q(s, a),$$

for each state s .

Proof. To analyze the behavior of Q-ALIGN DT as $\text{RTG}_{\text{tgt}} \rightarrow R^*$, we focus on the components of the loss that are sensitive to the target return. For a fixed state s , the alignment actor loss at R^* is defined as:

$$\begin{aligned} L_{\text{total}}(s, R^*) &= \mathbb{E}_{a \sim \mathcal{D}_{R^*}(s)} \|\pi_{R^*}(s) - a\|^2 + \mathbb{E}_{\text{RTG} \sim \mathcal{D}(s)} \mathbb{I}(Q(s, \pi_{\text{RTG}}(s)) > Q(s, \pi_{R^*}(s))) \\ &= \underbrace{\mathbb{E}_{a \sim \mathcal{D}_{R^*}(s)} \|\pi_{R^*}(s) - a\|^2}_{\text{supervised regression term}} + \underbrace{\mathbb{E}_{a' \sim \mathcal{D}(s)} \mathbb{I}(Q(s, a') > Q(s, \pi_{R^*}(s)))}_{\text{ranking term}}, \end{aligned}$$

where $\mathbb{I}(\cdot)$ is the indicator function. The second equality holds under the assumption that the model successfully minimizes the supervised loss within the data distribution, such that $\pi_{\text{RTG}}(s) \approx a'$ for $(s, a', \text{RTG}) \sim \mathcal{D}$.

Remark B.2. The assumption $\pi_{\text{RTG}}(s) \approx a'$ can be satisfied alongside L_{Align} because the two objectives are compatible: L_{SL} constrains the policy at discrete dataset points, while L_{Align} only enforces directional monotonicity with respect to RTG. Since the alignment loss vanishes once the ordering is satisfied, it does not conflict with the supervised targets. In the high-dimensional policy space, a policy exists that simultaneously respects L_{SL} and the global monotonic structure of L_{Align} , assuming the dataset is consistent.

In the high-RTG regime where R^* exceeds the return of any trajectory in the dataset, the supervised regression term vanishes as $\mathcal{D}_{R^*}(s) = \emptyset$. The objective thus simplifies to the expectation of the indicator function:

$$L_{\text{total}}(s, R^*) = \sum_{a' \in \mathcal{A}_{\text{data}}(s)} p_{\mathcal{D}}(a' | s) \cdot \mathbb{I}(Q(s, a') > Q(s, \pi_{R^*}(s))).$$

Since the empirical probability $p_{\mathcal{D}}(a' | s)$ is strictly positive for all $a' \in \mathcal{A}_{\text{data}}(s)$, the non-negative sum reaches its global minimum of 0 if and only if:

$$Q(s, \pi_{R^*}(s)) \geq Q(s, a'), \quad \forall a' \in \mathcal{A}_{\text{data}}(s).$$

This directly implies $Q(s, \pi_{R^*}(s)) \geq \max_{a' \in \mathcal{A}_{\text{data}}(s)} Q(s, a')$. Given that the dataset covers all state-action pairs, the global minimizer satisfies:

$$\pi_{R^*}^{\text{Align}}(s) = \arg \max_{a \in \mathcal{A}_{\text{data}}(s)} Q(s, a).$$

This completes the proof. $\square$

Lemma B.2. Assume the dataset fully covers all state-action pairs, consider an idealized version of QT where the conditioning space is extended to $R \rightarrow \infty$. Let π^{QT} denote the policy that minimizes the QT actor loss. Then, for the maximal target return R^* , π^{QT} outputs an action

$$a^* = \arg \max_{a \in \mathcal{A}_{\text{data}}(s)} Q(s, a),$$

for each state s .

Proof. For a fixed state s , the full QT actor loss is

$$L_{\text{QT}}(s, R^*) = \underbrace{\mathbb{E}_{a \sim \mathcal{D}_{R^*}(s)} \|\pi_{R^*}(s) - a\|^2}_{\text{supervised regression term}} - \underbrace{Q(s, \pi_{R^*}(s))}_{\text{Q-term}}.$$

When the target return R^* is larger than any trajectory in the dataset, the supervised regression term vanishes because $\mathcal{D}_{R^*}(s)$ is empty. The remaining loss reduces to

$$L_{\text{QT}}(s, R^*) = -Q(s, \pi_{R^*}(s)),$$

which is minimized by choosing

$$\pi_{R^*}(s) = \arg \max_{a \in \mathcal{A}_{\text{data}}(s)} Q(s, a).$$

Thus, for the maximal target return, π^{QT} outputs the action with the highest Q-value among all dataset actions, which concludes the proof. $\square$

Proof of Theorem 5.2. By combining the results of Lemma B.1 and Lemma B.2, we observe that both policies, π^{QT} and π^{Align} , satisfy the same optimality condition in the high-RTG limit. Specifically, since both objectives lead to the selection of the greedy action

$$a^* = \arg \max_{a \in \mathcal{A}_{\text{data}}(s)} Q(s, a) \quad (10)$$

for any given state s , the behavior of Q-ALIGN DT is theoretically equivalent to that of an idealized QT in this regime. This concludes the proof. $\square$

B.3. Proof of Theorem 5.3

To simplify the analysis, we consider *support-constrained optimality*, where all state-action pairs (s, a) are restricted to the support of the offline dataset $\mathcal{D}$, and, for theoretical analysis, we assume exact critic updates. When $\Delta\text{RTG} = 0$, the critic Bellman update evaluates the unperturbed RTG-conditioned policy within the dataset support, yielding a conservative, behavior-aligned value estimate. Consequently, the combination of the supervised learning loss and the unperturbed critic keeps the policy anchored near the behavior policy β . We then focus on the **high-RTG regime** where the perturbation ΔRTG is sufficiently large to steer the policy toward high-value regions. Following IQL (Kostrikov et al., 2022), we define the support-constrained optimal state-action value function Q^* and the corresponding optimal policy π^* as

$$\begin{aligned} Q^*(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} \left[\max_{a' \in \mathcal{A}: \pi_\beta(a'|s') > 0} Q^*(s', a') \right], \\ \pi^*(s) &= \arg \max_{a \in \mathcal{A}: \pi_\beta(a|s) > 0} Q^*(s, a), \end{aligned}$$

where $\mathcal{A}$ is the action space and π_β is the behavior policy of the offline dataset $\mathcal{D}$.

To proceed, we first establish the following lemma:

Lemma B.3. *Let Q_m be the action-value function of some policy π . Let π_m be the policy obtained by minimizing the alignment actor loss on Q_m . Let Q_{m+1} be the action-value function obtained by minimizing the TD loss (7) with $\Delta\text{RTG} = R^*$.*

Then for every (s, a) we have

$$Q_{m+1}(s, a) \geq Q_m(s, a).$$

Moreover, if there exists some state s where $\pi_m(s)$ strictly improves the one-step target, then the inequality is strict for at least one state-action pair.

Proof sketch. For notational simplicity, we use $\tilde{\pi}_m$ to denote the policy induced by π_m under a return perturbation $\Delta\text{RTG} = R^*$. We prove the claim by induction on the remaining horizon.

For the terminal timestep H , where no bootstrapping is applied, we have

$$Q_{m+1}(s_H, a_H) = Q_m(s_H, a_H) = r(s_H, a_H).$$

Assume the statement holds for all timesteps $t + 1, \dots, H$. Consider a state-action pair (s_t, a_t) at timestep t :

$$\begin{aligned} Q_{m+1}(s_t, a_t) &= r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim P(\cdot|s_t, a_t)} \left[Q_{m+1}(s_{t+1}, \tilde{\pi}_m(s_{t+1})) \right] \\ &\geq r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim P(\cdot|s_t, a_t)} \left[Q_m(s_{t+1}, \tilde{\pi}_m(s_{t+1})) \right] \\ &\geq r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim P(\cdot|s_t, a_t)} \left[Q_m(s_{t+1}, \pi(s_{t+1})) \right] \\ &= Q_m(s_t, a_t), \end{aligned}$$

where the first inequality follows from the induction hypothesis, and the second inequality follows from Lemma B.1. This completes the induction and proves $Q_{m+1} \geq Q_m$, with strict inequality for at least one state-action pair if π_m strictly improves the one-step target. $\square$

Proof of Theorem 5.3. By Lemma B.3, the Q-ALIGN DT update ensures $\{Q_m\}$ is monotonically non-decreasing: $Q_{m+1} \geq Q_m$ on $\mathcal{D}$. Given bounded rewards and finite horizon, the sequence $\{Q_m\}$ converges pointwise to a limit $\bar{Q}$ by the Monotone Convergence Theorem.

We show $\bar{Q} = Q^*$ on $\mathcal{D}$ by contradiction. Suppose $\bar{Q}$ is not optimal; then there exists $(s, a) \in \mathcal{D}$ such that:

$$\bar{Q}(s, a) < r(s, a) + \gamma \mathbb{E}_{s' \sim P} \left[\max_{a' \in \mathcal{D}(s')} \bar{Q}(s', a') \right]. \quad (11)$$

By Lemma B.1, under a sufficiently large perturbation δ , the induced policy $\tilde{\pi}$ (derived from the converged policy π_θ) selects greedy actions w.r.t. $\bar{Q}$: $\tilde{\pi}(s) = \arg \max_{a' \in \mathcal{D}(s)} \bar{Q}(s, a')$. Substituting this into (11) yields:

$$\bar{Q}(s, a) < r(s, a) + \gamma \mathbb{E}_{s' \sim P} [\bar{Q}(s', \tilde{\pi}(s'))]. \quad (12)$$

However, upon convergence, $\bar{Q}$ must satisfy the Bellman equation corresponding to the induced policy $\tilde{\pi}$:

$$\bar{Q}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P} [\bar{Q}(s', \tilde{\pi}(s'))],$$

which directly contradicts (12). Therefore, $\bar{Q}$ satisfies the Bellman optimality equation on $\mathcal{D}$, implying $\bar{Q} = Q^*$. Consequently, the sequence $\{\tilde{\pi}_m\}$ converges to an optimal policy $\pi^*(s) \in \arg \max \bar{Q}(s, a)$ restricted to the dataset. $\square$

C. Experiments on halfcheetah-vel

Transferring a model trained on the halfcheetah environment and deploying it directly on halfcheetah-vel is challenging, as the two environments differ in several important ways. We summarize the main challenges as follows:

- **Different Reward Functions.** Although both environments include a forward-velocity term and a control cost, their reward definitions differ significantly. In halfcheetah, the forward reward is simply the current velocity. In contrast, halfcheetah-vel defines the forward reward as

$$-|v_t - v_{\text{target}}|,$$

where v_t is the current velocity and v_{target} is the target velocity.

- **Different Horizons.** halfcheetah uses a horizon of 1000 steps, whereas halfcheetah-vel terminates after 200 steps. This discrepancy affects the RTG interpretation learned during training.
- **Different Observation Dimensions.** Compared to halfcheetah, the halfcheetah-vel environment further includes a 3-dimensional absolute position in its observations.

For fair comparison on halfcheetah-vel, we directly deploy the model trained on halfcheetah, but apply several modifications to the inputs:

- **Reward Mismatch.** To maintain consistency with the training setup, we re-compute the reward using the current forward velocity, updating the RTG as:

$$\bar{r}_t = v_t + c_t, \quad \text{RTG}_{t+1} = \text{RTG}_t - \bar{r}_t,$$

where c_t is the control cost from halfcheetah-vel. Note that although this modification is applied when updating the RTG, we report results using the original reward definition of halfcheetah-vel for all transferred methods.

- **Horizon Mismatch.** We continue to construct the RTG token using the original horizon of 1000 (i.e., $1000 \times v_{\text{target}}$), while the environment naturally truncates at 200 steps.
- **Observation Mismatch.** We simply discard the extra 3 dimensions in halfcheetah-vel, and rely on the RTG token to convey velocity-related information.

Table C.6. Effect of the number of calibration samples on zero-shot transfer performance.

Calibration Samples	5	10	20	40
Q-Align DT	-158.5	-155.7	-152.3	-142.3

Even after applying these adjustments, selecting an appropriate initial RTG for deployment on `HalfCheetah-vel` remains challenging. The main difficulty lies in estimating the cumulative control cost required to compute the RTG corresponding to a given target velocity: since the control cost depends on the induced trajectory, the desired RTG cannot be determined analytically and must be calibrated empirically at deployment time. Moreover, since the model is trained with a 1000-step horizon, it is allowed to defer RTG consumption toward later timesteps; empirically, we observe that the learned policy can exhibit such behavior (as illustrated in Figure 4), which becomes infeasible under the 200-step evaluation horizon.

To address this mismatch, we define a base RTG as $1000 \times v_{\text{target}}$ and empirically calibrate the RTG values. Specifically, we sample 40 RTG candidates around this base value for the maximum and minimum target velocities to identify the corresponding optimal RTGs, denoted as RTG_{max} and RTG_{min} . For intermediate target velocities v , we assume a smooth and approximately monotonic relationship between RTG and the resulting velocity, and linearly interpolate the RTG as

$$\text{RTG}(v) = \text{RTG}_{\text{min}} + \frac{v - v_{\text{min}}}{v_{\text{max}} - v_{\text{min}}} (\text{RTG}_{\text{max}} - \text{RTG}_{\text{min}}).$$

All methods are evaluated using this procedure independently to ensure a fair comparison.

The effectiveness of linear interpolation demonstrates both the controllability and alignment of Q-ALIGN DT. In standard CSMs, the mapping from RTG to realized behavior is often inconsistent, leading to poor alignment with the target velocity in this setting. In contrast, Q-ALIGN DT learns a smooth and approximately linear relationship between RTG and behavior. This allows us to anchor the policy at only the two endpoint velocities and reliably interpolate for intermediate targets, handling the environment shift without the need for additional RTG tuning, a strategy that fails for standard CSMs.

To understand the sensitivity of Q-ALIGN DT to the number of RTG candidates, we conduct experiments with different calibration budgets and evaluate on `halfCheetah-vel`. As shown in Table C.6, zero-shot transfer performance degrades gradually rather than collapsing as the number of calibration samples is reduced. Even with substantially fewer calibration samples, Q-ALIGN DT remains much stronger than the other CSM-based baselines evaluated with 40 samples, as shown in Table 5.

D. Implementation Details

D.1. Model Architecture

CSM Architecture. We adopt a Transformer-based policy following the sequence-conditioned structure in Chen et al. (2021). To enhance the local temporal consistency of control signals, we insert a lightweight 1D convolution layer after the projection matrices W_k, W_q, W_v . Specifically, for each layer, we compute:

$$k_\ell = \text{Conv}(W_k x_\ell), \quad q_\ell = \text{Conv}(W_q x_\ell), \quad v_\ell = \text{Conv}(W_v x_\ell).$$

The convolution window size is set to $w = 6$, enabling each output to incorporate information from a local temporal neighborhood. To preserve causality, we apply *causal left padding* by padding the input sequence with $w - 1 = 5$ zeros on the left, ensuring that the output at timestep t depends only on inputs at or before t . This prevents future information leakage and maintains the temporal causality required for sequence modeling. As demonstrated in our ablation studies, this modification does not dictate the peak performance.

Prediction Heads. Following the design in Hu et al. (2024), the model performs prediction sequentially at each timestep i . Specifically, the hidden representation of the RTG token rtg_i is mapped through a linear head to reconstruct the current state $\hat{s}_i$, and the representation of the state token s_i is mapped through a separate head to predict the corresponding action $\hat{a}_i$. These architectural details are summarized in Table D.7.

Q-Function. For the Q -function, we adopt a 3-layer MLP with a hidden dimension of 256.

D.2. Q Pretraining

For stability, we pretrain the Q -function before training the CSM. To isolate the effect of the proposed alignment method, we adopt a simple Double Q -learning update (Hasselt et al., 2016) in all experiments, except for the AntMaze environments, where IQL (Kostrikov et al., 2022) is used.

Specifically, for Q networks Q_{ψ_1}, Q_{ψ_2} and target networks $Q_{\psi'_1}, Q_{\psi'_2}$, the pretraining loss is

$$L_{\text{pretrain}} = \sum_{i=t-k+1}^t \sum_{m=1}^2 \left(Q_{\psi_m}(s_i, a_i) - y_i \right)^2, \quad y_i = r_i + \gamma \min_{m=1,2} Q_{\psi'_m}(s_{i+1}, a_{i+1}),$$

where $\gamma = 0.99$. While RTG typically represents an undiscounted return ($\gamma = 1$) and we follow this standard practice for RTG conditioning, we employ a discounted Q -function ($\gamma = 0.99$) solely as a guidance signal during pretraining and alignment. This design choice is empirically robust and consistent with prior successful architectures (Hu et al., 2024; Kim et al., 2024b; Yan et al., 2024). Importantly, since our alignment loss relies exclusively on the **relative ordering** rather than absolute magnitude, the scale discrepancy introduced by the discount factor does not affect the directional gradient. As long as the discounted Q -function preserves monotonic preference over returns, the structural regularization remains theoretically sound and avoids optimization inconsistency.

For AntMaze, standard Bellman updates often fail to propagate values across long horizons in offline settings, leading to vanishing gradients and uninformative value estimates. We therefore use IQL for pretraining, with an expectile of 0.8.

Table D.7. Architecture of Q-ALIGN DT

Parameter	Value
Number of layers	4
Attention heads	4
Conv window size	6
Embedding dimension	256
Batch size	256
Dropout	0.1
Learning rate	3e-4
Target update rate	0.005
Nonlinearity	GELU (Hendrycks & Gimpel, 2016)
Optimizer	Adam (Kingma & Ba, 2014)

D.3. Baseline Details

We evaluate Q-ALIGN DT against a comprehensive set of baselines, including *traditional offline RL methods*: IQL (Kostrikov et al., 2022), TD3+BC (Fujimoto & Gu, 2021), and CQL (Kumar et al., 2020); and *sequence modeling approaches*: DT (Chen et al., 2021), DC (Kim et al., 2024a), RVS (Emmons et al., 2022), CGDT (Wang et al., 2024), LSDT (Wang et al., 2025), DM (Zheng et al., 2025), RADT (Tanaka et al., 2025), QT (Hu et al., 2024), and QCS (Kim et al., 2024b).

For most tasks, we report the performance directly from their original publications. However, for the AntMaze environment, some baselines such as QT (Hu et al., 2024) originally reported results on the v0 version. To ensure a fair comparison on the current standard benchmark, we re-evaluate these methods on the v2 environment using their official codebases. For the specific evaluation of alignment properties, we re-train and evaluate DT, DC, RADT, QT, and QCS using their respective official implementations to ensure consistency in our experimental setup.

D.4. Hardware and Environment Configuration

All experiments were conducted on a NVIDIA L40S GPU with 48GB of HBM2 memory. The CPU is an Intel(R) Xeon(R) Gold 6430.

For the software environment, we used:

- d4rl 1.1, gym 0.18, mujoco 2.0.2

- PyTorch 2.4, CUDA 12.1
- Operating system: Red Hat Enterprise Linux 9.6
- Environment managed with `conda` and Python 3.8

D.5. Hyperparameters

We report the hyperparameters of Q-ALIGN DT for the selected tasks in Table D.8. Below, we describe the key components: the noise scale σ_e , the weight of the Q -alignment loss λ_e , and the RTG perturbation magnitude ΔRTG . Following the standard protocol of Decision Transformer (Chen et al., 2021), we scale the target RTG by a factor of 1/1000 for all D4RL Gym environments. Accordingly, the perturbation scale ΔRTG and the noise magnitude σ_e reported in this paper are defined with respect to these normalized RTG values.

The choice of σ_e is primarily influenced by the reward scale of each environment. The coefficient λ_e varies across datasets because it depends on both the dataset distribution and the stability of the environment during Q -function updates.

The perturbation value ΔRTG is introduced during the Q -function update so that the learned critic reflects the value of relatively high-reward policies within the policy family of Q-ALIGN DT. Ideally, we would like it to approximate the value of the best policy in this family, i.e., by letting $\Delta\text{RTG} \rightarrow \infty$. However, in practice this may lead to out-of-distribution behavior and destabilizes training. Therefore, we choose ΔRTG according to the stability characteristics of each environment.

Table D.8. Hyperparameters of Q-ALIGN DT across different environments. σ_e controls the scale of RTG perturbation noise; λ_e is the weight of the Q -alignment loss; ΔRTG specifies the RTG perturbation used for Q -function updates.

Environment / Dataset	σ_e	λ_e	ΔRTG
walker2d-medium-replay-v2	10	0.3	2
halfcheetah-medium-replay-v2	15	5	5
hopper-medium-replay-v2	10	0.3	0.5
walker2d-medium-v2	10	0.3	1
halfcheetah-medium-v2	15	5	10
hopper-medium-v2	10	0.3	1
walker2d-medium-expert-v2	10	0.3	10
halfcheetah-medium-expert-v2	15	1	10
hopper-medium-expert-v2	10	0.3	5
antmaze-umaze-v2	5	100	1
antmaze-umaze-diverse-v2	5	100	1
antmaze-medium-play-v2	5	100	1
antmaze-medium-diverse-v2	5	100	1

Specifically, when sampling RTG perturbations δ in Antmaze environment, we employ a half-normal distribution, $\delta = |\epsilon|$, $\epsilon \sim \mathcal{N}(0, \sigma_e^2)$. This ensures that the perturbations are positive, providing directional guidance toward the goal in sparse-reward navigation tasks, while negative perturbations, which are uninformative in this setting, are avoided.

We do not perform extensive hyperparameter sweeps, as our goal is to evaluate the robustness and controllability of the proposed method rather than optimize peak performance. Accordingly, we largely share hyperparameters across environments within the same domain, making only minor adjustments when necessary for training stability. Despite this minimal tuning, Q-ALIGN DT consistently outperforms baselines, indicating that the observed gains arise from improved alignment rather than hyperparameter optimization. For AntMaze, we fix a larger λ_e across all variants to accommodate the sparse-reward structure and different reward scale.

D.6. Alignment Calculation

For evaluating alignment performance, we sweep the target RTG from the minimum to the maximum return in each dataset with a step size of 100. For each RTG, we run 30 rollouts and compute the average D4RL score. The root mean squared deviation between the achieved scores and the target scores is used as the alignment metric. The minimum and maximum returns of each dataset are listed in Table D.9.

This evaluation is similar to prior work (Tanaka et al., 2025); however, unlike RADT which evaluates only 7 RTG values per dataset, our method considers at least 32 distinct RTG values (in Hopper), providing a finer-grained assessment of policy alignment across the entire return spectrum.

Table D.9. Maximum and Minimum return of each dataset

Dataset	Maximum return	Minimum return
Hopper-v2	3234.3	-20.3
Walker2d-v2	4592.3	1.6
Halfcheetah-v2	12135.0	-280.2

E. More Experiment Results

E.1. More Ablation Experiments

In this section, we provide a detailed ablation analysis of the core components in Q-ALIGN DT.

Effect of Fixing the Q -function. We conduct an ablation in which the Q -function is fixed after pretraining and is no longer updated jointly with the actor. As shown in Table E.10, fixing the Q -function leads to a clear degradation in the best achievable performance, as the static critic fails to provide informative signals to improve the actor beyond the behavior policy. Furthermore, alignment error increases accordingly, indicating that co-training is essential for maintaining reward-sensitive behavior.

Notably, most of the degradation occurs under low-RTG conditions. During inference, when conditioned on a relatively low target RTG, the model trained with a fixed Q -function often collapses (e.g., falls) in the early timesteps. This early failure causes the realized return to approach zero, falling far below the intended RTG target and resulting in a breakdown of alignment (see Section E.5 for further results).

Table E.10. Effect of fixing the Q -function. RMSE ($\downarrow$) denotes the alignment error, while Perf. ($\uparrow$) denotes the overall performance (D4RL normalized score). Results are averaged over three random seeds.

Dataset	Align. RMSE $\downarrow$		Perf. $\uparrow$	
	Q-ALIGN DT	Fix Q	Q-ALIGN DT	Fix Q
Medium-Replay	11.31	24.45	86.45	80.32
Medium	9.19	28.04	87.70	79.72
Medium-Expert	6.80	16.90	111.37	106.83

Asymmetric vs. Symmetric Indicator Functions. We investigate the necessity of the **asymmetric indicator function** by considering a symmetric variant:

$$\mathbb{I}_{\text{sym}} = \begin{cases} 1, & \text{if } \text{sgn}(\delta)(Q_\psi(s_i, \hat{a}_i^\delta) - Q_\psi^\perp(s_i, \hat{a}_i)) < 0, \\ -1, & \text{otherwise.} \end{cases}$$

Unlike our original formulation, which only penalizes constraint violations, this variant explicitly encourages the actor to further maximize the value gap even when the ranking is already correct.

As shown in Table E.11, this “active alignment” approach does not generally improve performance. On the contrary, it significantly increases training instability and even leads to behavioral collapse in environments like Hopper and Walker2d. This suggests that the primary role of the alignment loss should be *correcting* misaligned rankings rather than continuously pushing actions beyond the critic’s reliable regions, which may introduce detrimental gradient noise.

Sensitivity to the Alignment Loss Form. We further study the sensitivity of Q-ALIGN DT to the functional form of the alignment loss. Specifically, we replace the default linear penalty with a Squared Penalty,

$$(Q_\psi(s_i, \hat{a}_i^\delta) - Q_\psi^\perp(s_i, \hat{a}_i))^2,$$

which imposes a quadratic cost on directional inconsistencies.

As shown in Table E.11, this variant yields comparable performance across all tasks, with only minor deviations from the default formulation. Empirically, introducing higher-order penalties does not yield consistent improvements, which leads us to adopt the simplest formulation, viewable as a first-order approximation of the value difference under RTG perturbations.

Table E.11. Ablation of the alignment loss structure. We compare the default Q-ALIGN DT with its symmetric and squared (L2) variants.

Dataset	Q-ALIGN DT (Ours)	Symmetric Penalty	Square Penalty
Hopper-medium	102.5	42.9 (collapse)	98.7
Walker2d-medium	95.1	14.3 (collapse)	93.4
Halfcheetah-medium	65.5	63.4	68.7

Sensitivity to RTG σ_e . We evaluate the sensitivity of Q-ALIGN DT to the perturbation magnitude σ_e , which is defined relative to the normalized RTG range. Table E.12 reports the performance on the `halfcheetah-medium` task across a wide range of σ_e values.

The results indicate that Q-ALIGN DT is highly robust to the choice of σ_e within a reasonable range (from 1 to 50). Even a large perturbation ($\sigma_e = 50$) does not lead to performance collapse, suggesting that the directional signal remains informative. By contrast, an excessively small perturbation (e.g., $\sigma_e = 0.5$) limits the actor’s ability to explore the value landscape, resulting in suboptimal alignment.

Table E.12. Sensitivity analysis of the perturbation scale σ_e on `halfcheetah-medium`. The noise is applied to the normalized RTG (scaled by 1/1000). Q-ALIGN DT exhibits strong robustness across an order of magnitude of noise levels.

σ_e	0.5	1.0	5.0	10.0	15.0	50.0
D4RL Score	43.3	66.7	65.7	65.3	65.5	64.7

Sensitivity to RTG λ_e . We further evaluate the sensitivity of Q-ALIGN DT to the alignment loss weight λ_e . As shown in Table E.14, we report the performance on `halfcheetah-medium` across a wide spectrum of weights.

The results reveal a clear phase transition: at $\lambda_e = 0$, where the directional guidance is absent, the model achieves a score of only 42.84, consistent with standard supervised learning baselines. However, with even a small weight ($\lambda_e = 1.0$), the performance surges to 61.7. Crucially, for $\lambda_e \geq 3.0$, the performance enters a stable plateau, with scores remaining consistently above 64.0 regardless of the specific weight chosen. While the peak performance reaches 68.2 at $\lambda_e = 100.0$, the marginal gains across the 3.0 to 100.0 range suggest that Q-ALIGN DT is highly robust to this hyperparameter. This insensitivity allows for a “plug-and-play” deployment without the need for exhaustive, task-specific fine-tuning.

Sensitivity to Q Function Accuracy We investigate the sensitivity of Q-ALIGN DT to inaccuracies in the learned Q function. Specifically, we perturb the target critic $Q_{\psi'}$ by injecting Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma_n^2)$ to simulate potential estimation errors.

We conduct experiments on `halfcheetah-medium` and `walker2d-medium` with noise scales $\sigma_n \in \{1, 10, 50\}$, and report the results in Table E.13. Notably, Q-ALIGN DT exhibits remarkable resilience, maintaining stable performance even under substantial perturbations. This indicates that the method is insensitive to the exact accuracy of the Q -function. We attribute this robustness to the alignment objective, which emphasizes the *relative ordering* and *directional consistency* of Q -values rather than their absolute magnitudes. As long as the critic preserves coarse rank-ordering of actions, the alignment loss provides sufficient guidance for effective policy extraction, making the learning process tolerant to estimation noise.

E.2. Extended Generalization on `HalfCheetah-Vel`

While the standard `halfcheetah-vel` task focuses on target velocities within the $[0, 3]$ range, our model leverages the diverse transitions in the `medium-expert` dataset to acquire a behavioral repertoire capable of reaching velocities exceeding 10. To fully explore the limits of its controllability, we evaluate Q-ALIGN DT on extended target velocities that far exceed those typically used in Meta-RL benchmarks.

Table E.13. Sensitivity analysis of the value function. We report the normalized D4RL scores under varying noise scales σ_n .

σ_n	0	1.0	10	50.0
halfcheetah-medium	65.5	65.1	64.4	63.4
walker2d-medium	95.1	91.1	91.8	90.7

Table E.14. Sensitivity analysis of the perturbation scale λ_e on halfcheetah-medium.

λ_e	0	1.0	3.0	5.0	10.0	100.0
D4RL Score	42.84	61.7	64.42	65.5	65.3	68.2

We follow the evaluation protocol in Section C, extending the episode horizon to 1000 steps to allow sufficient time for the agent to accelerate to these higher targets. Since the original halfcheetah-vel dataset is restricted to low-velocity samples, we use CSMs trained on the broader halfcheetah-medium-expert dataset as baselines. As reported in Table E.15, Q-ALIGN DT demonstrates a remarkably wide effective range of controllability, accurately tracking high-velocity targets even when trained under a singular and straightforward reward objective (i.e., where reward simply scales with velocity).

Table E.15. **Performance under high target velocities on HalfCheetah-vel.** Maximum episode return is reported for each target velocity (horizon = 1000).

Target Vel.	DC	DT	RADT	QT	Q-ALIGN DT
4.0	-1749.01	-1744.94	-1709.06	-1899.36	-1289.17
5.0	-1332.75	-1573.92	-2452.64	-2716.32	-1246.46
6.0	-1061.06	-1094.22	-1085.82	-2247.88	-1203.69
7.0	-3666.29	-3627.29	-1026.00	-3967.86	-1441.51
8.0	-3519.51	-4101.90	-2100.74	-4781.54	-2366.28
9.0	-2850.36	-3249.43	-3888.88	-4188.40	-2422.39
10.0	-2162.07	-2412.16	-3261.80	-3528.83	-1493.27
Mean	-2334.44	-2543.41	-2217.85	-3332.88	-1637.54

E.3. Behavioral Analysis on Ant

We also evaluate Q-ALIGN DT on ant-medium to examine whether the agent can modulate its behavior under different target RTGs. As shown in Figure E.5, the agent demonstrates similar trends to halfcheetah: higher target RTGs induce faster locomotion, while lower RTGs lead to more conservative movement. These results suggest that, even when trained only on scalar returns, the actor can learn distinct behaviors that are controllable via the RTG tokens, indicating that the structured mapping from RTG to actions generalizes across different environments.

E.4. Additional Experiments

We first show the last iteration performance of DT, QT and Q-ALIGN DT in Table E.16

We report a detailed ablation of the architectural change and the alignment objective in Table E.17. To make the attribution clearer, we compare vanilla DT, DT with 1D convolution, DT with the alignment loss, and Q-Align DT in terms of alignment MSE. As shown in Table E.17, 1D convolution alone slightly reduces the alignment MSE, while the alignment loss provides the dominant improvement. Combining both components achieves the lowest alignment MSE, indicating that the main gain comes from the proposed alignment objective rather than the architectural modification alone.

We further report the training cost of DT, QT, and Q-ALIGN DT in Table E.18, where each method is trained for 100 epochs with 1000 steps per epoch. QT and Q-ALIGN DT are slower than DT because both use an additional Q-function. Compared with QT, Q-ALIGN DT only adds extra forward computation for perturbed-action generation, without introducing additional

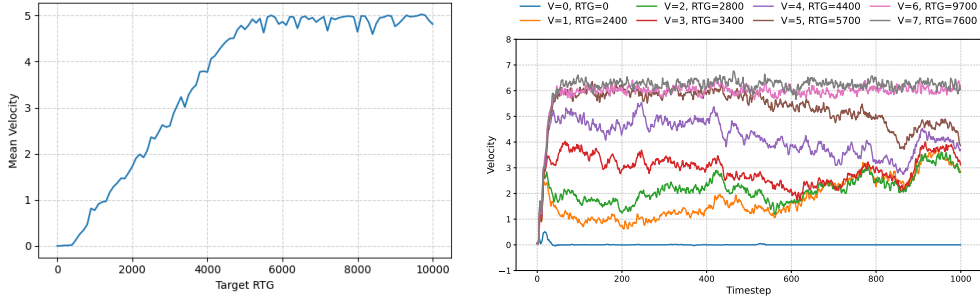


Figure E.5. Behavioral modulation of the agent on `ant-medium` under varying target RTGs. (Left) The relationship between target RTG and realized mean velocity; (Right) Velocity trajectories across different RTG levels averaged over 30 rollouts.

Table E.16. Last-iteration performance comparison on D4RL Gym tasks.

Task	DT	QT	Q-Align DT
HalfCheetah-medium	42.20	50.5	64.8 ± 0.7
HalfCheetah-medium-replay	38.91	48.4	52.3 ± 1.5
HalfCheetah-medium-expert	91.55	94.0	96.6 ± 1.7
Hopper-medium	65.10	63.2	98.7 ± 3.4
Hopper-medium-replay	81.77	99.4	99.9 ± 1.3
Hopper-medium-expert	110.44	108.2	112.5 ± 1.4
Walker2d-medium	67.63	86.5	92.5 ± 2.9
Walker2d-medium-replay	59.86	88.5	100.8 ± 1.8
Walker2d-medium-expert	107.11	110.5	115.8 ± 1.9
Sum	664.6	749.2	828.8

backward-pass overhead. As a result, its practical training cost remains comparable to QT.

To better understand the different behavior of Q-ALIGN DT on Gym and AntMaze tasks, we further compute the Spearman correlation between predicted Q -values and realized returns, as shown in Table E.19. The correlation is much lower on AntMaze than on dense-reward Gym tasks (e.g., 0.94 on Hopper-medium vs. 0.26 on AntMaze-umaze-diverse). This suggests that the Q -guided alignment signal is substantially weaker in sparse-reward environments, especially on the diverse split. We view this phenomenon as a limitation of Q-ALIGN DT in sparse-reward settings and leave further improvement to future work.

E.5. Further Analysis on Alignment Stability

We first present the complete alignment curves for Q-ALIGN DT across all nine Gym tasks in Figure E.6, demonstrating consistently robust alignment.

Beyond overall performance, we examine scenarios in which alignment degrades, specifically under a fixed critic or when the RTG offset is near zero. As shown in Figure E.7, when the Q -function is fixed, the agent’s performance under low-RTG targets effectively collapses to zero. A stable response only emerges once the target RTG exceeds a certain threshold, indicating a sharp transition in behavior. In Figure E.8, we further report the agent’s velocity at each timestep, which shows that the misalignment in the low-RTG regime is caused by early timesteps collapse.

We further examine the impact of RTG offsets in Figure E.9. Notably, even with synchronized training, a near-zero offset replicates the failure mode of the fixed-critic baseline. This is because a sufficient ΔRTG is essential to ensure that the Q -function evaluates the better-performing policies within the evolving policy family. Without this positive offset, the alignment objective is guided by value estimates of mediocre or failing behaviors. Consequently, the policy lacks the structural guidance needed to remain stable under low-RTG targets, leading to the observed performance collapse where realized returns drop to zero.

Table E.17. Ablation study on the effect of 1D convolution and the alignment loss, measured by alignment MSE. Lower is better.

Dataset	DT	DT + 1DConv	DT + Align Loss	Q-Align DT
Medium-Replay	20.32	18.07	13.37	11.31
Medium	25.33	20.32	14.22	9.19
Medium-Expert	18.53	19.43	10.70	6.80
Mean	21.39	19.27	12.76	9.12

Table E.18. Wall-clock training time under the same setting of 100 epochs and 1000 steps per epoch.

Method	DT	QT	Q-ALIGN DT
Training Time	~4 h 36 min	~6 h 7 min	~6 h 22 min

Table E.19. Spearman correlation between predicted Q -values and realized returns across environments.

Environment	Spearman Corr.
Hopper-medium	0.94
AntMaze-umaze	0.36
AntMaze-umaze-diverse	0.26

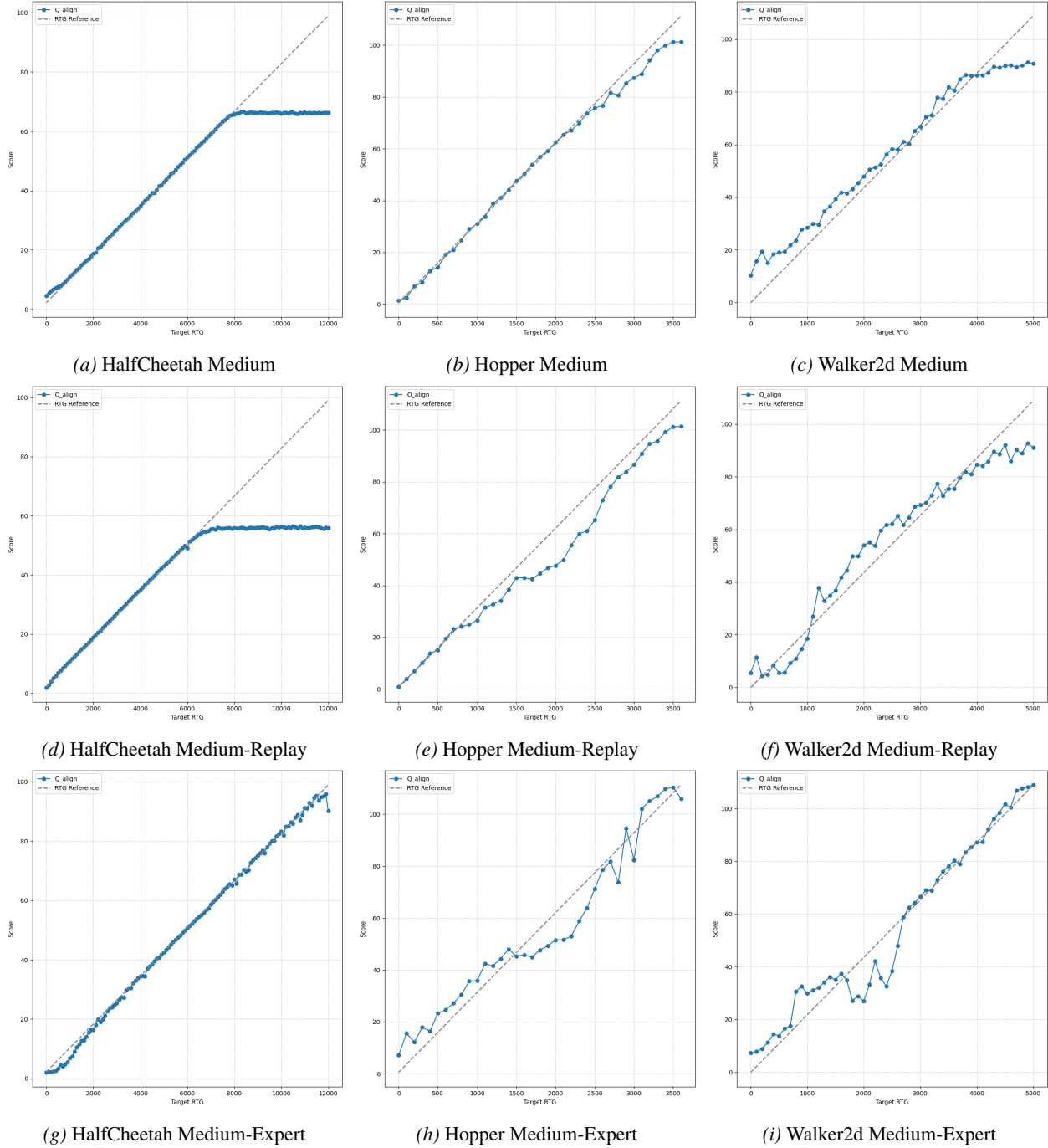


Figure E.6. Performance of Q-ALIGN DT across Gym locomotion tasks. Each row corresponds to one dataset type, and each column corresponds to one environment.

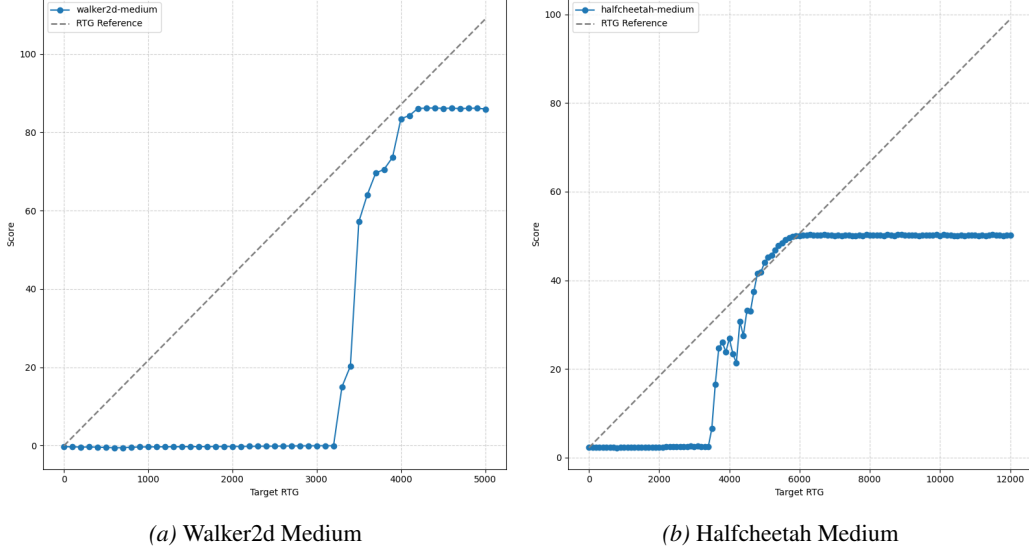


Figure E.7. Results of walker2d-medium and halfcheetah-medium when Q is fixed.

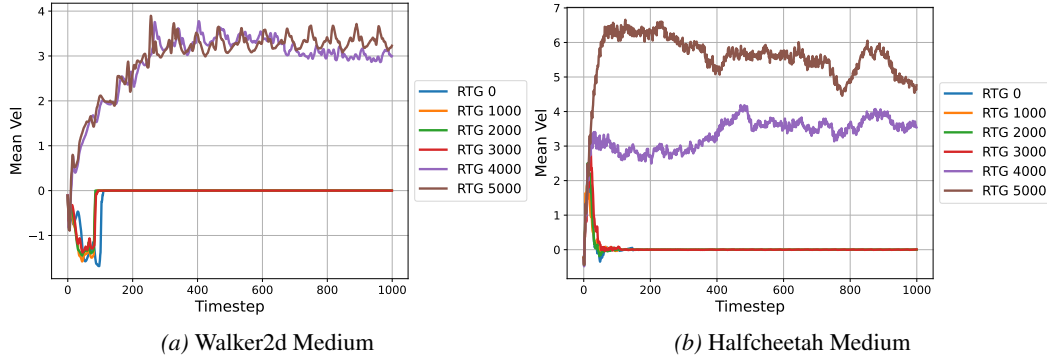


Figure E.8. Velocity on walker2d-medium and halfcheetah-medium with a fixed Q . At lower RTG targets, the model consistently **collapses** within the first few timesteps, rendering it **unresponsive** to RTG changes. A meaningful behavioral response to the conditioning only emerges after the RTG exceeds a critical threshold, allowing the model to escape early collapse.

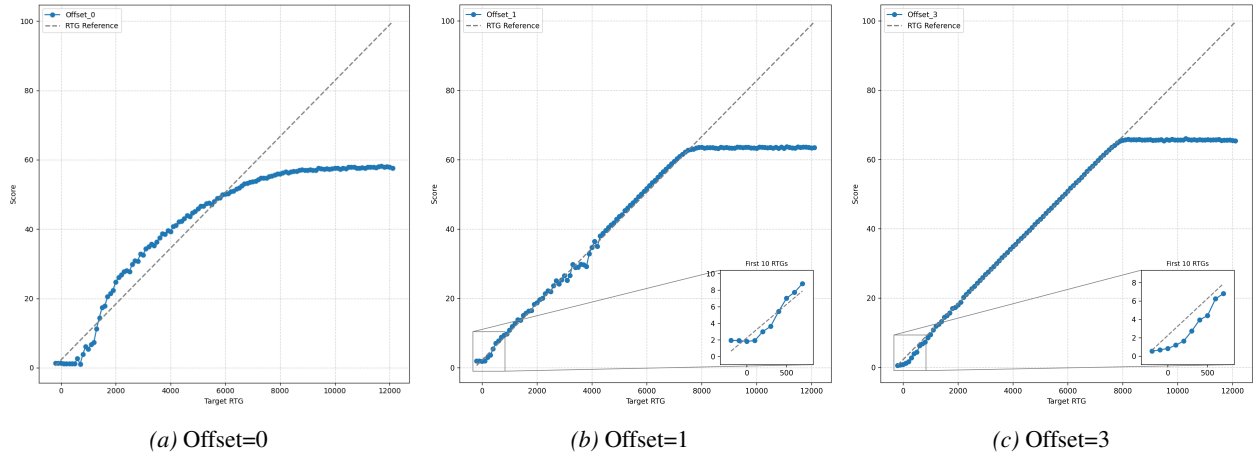


Figure E.9. Results halfcheetah-medium's alignment curve under different RTG offset.